

EWEMrl: A White-Box Secure Cipher with Longevity

Avik Chakraborti^{1,2}, Shibam Ghosh³, Takanori Isobe⁴ and Sajani Kundu^{1,5}

¹Institute for Advancing Intelligence, TCG CREST

²Academy of Scientific and Innovative Research (AcSIR)

³INRIA, Paris

⁴Osaka University, Japan

⁵Ramakrishna Mission Vivekananda Educational and Research Institute

avikchkrborti@gmail.com, shibam.math@gmail.com, takanori.isobe@ist.osaka-u.ac.jp,
sajani.kundu.89@tcgcrest.org

Abstract. We propose the first updatable white-box secure cipher, EWEMrl (Extended **WEM** with longevity against non-adaptive read-only adversaries), and its natural extension, EWEMxl (Extended **WEM** with longevity against executable adversaries), both based on **WEM** (White-box Even-Mansour), and both achieving longevity against non-adaptive read-only malware. The notion of *longevity*, introduced by Koike et al., addresses continuous code leakage and is stronger than *incompressibility*. While Yoroi claimed longevity, but was broken by Isobe and Todo. Given the prevalence of continuous leakage, developing such ciphers is crucial in white-box cryptography. Precisely, we have the following.

- We first present EWEMr (Extended **WEM** against non-adaptive read-only adversaries), a generalization of **WEM** (White-box Even-Mansour). **WEM** is the first (and possibly only) white-box cipher based on Even-Mansour (**EM**), replacing its key addition layer with a secret Sbox. EWEMr achieves a high space-hardness bound in the non-adaptive model, with a new generic proof strategy, but does not provide longevity. Instead, it serves as the base for EWEMrl.
- We also present EWEMx (Extended **WEM** against executable adversaries), which uses EWEMr as subroutines and achieves a high space-hardness bound in the stronger adaptive model. While EWEMx does not achieve longevity, it is the base design for EWEMxl.
- We next propose EWEMrl, that achieves longevity against non-adaptive read-only malware. None of the existing ciphers, such as SPNbox and SPACE, are designed for longevity. We show that EWEMrl ensures (against non-adaptive read-only adversaries) (1) longevity, (2) high space-hardness in both known-space and chosen-space settings, and (3) security against hybrid code-lifting attacks.
- Finally, we introduce EWEMxl, a natural extension of EWEMrl with a structure similar to EWEMx. EWEMxl achieves (2) and (3) in the stronger adaptive model while maintaining (1) in the same non-adaptive and read-only setting.

In summary, our proposals EWEMrl and EWEMxl provide longevity against non-adaptive read-only malware while ensuring security confidence in the black-box setting.

1 Introduction

Traditional cryptographic primitives protect data and keys under the assumption that attackers can only access the cipher’s input-output interface as a black-box. Attackers

may ask for known, chosen, or adaptively chosen queries to the black-box oracle either to recover the master key (stronger goal) or to distinguish the cipher from a random counterpart (weaker goal). However, this assumption fails in untrusted environments. White-box cryptography, introduced by Chow *et al.* in SAC 2002 [12, 13], safeguards secret data embedded in the software implementation of a block cipher. In the white-box model, the adversary has full access to the execution environment, including the cipher's internal states, unlike the black-box model, where such access is restricted. The attacker's goals include extracting the master key (key extraction) or compressing the implementation without losing functionality (incompressibility).

1.1 Real-World Usage of White-Box Cryptography

Chow *et al.* [12, 13] introduced White-box Cryptography, targeting Digital Rights Management (DRM) applications. With other software-based solutions such as *host card emulation* (HCE) replacing *Hardware Secure Elements* (HSE) in *Near Field Communication* (NFC) transactions, white-box cryptography has gained more significance in software-only security, adapting to complex systems like cloud services. The rise of advanced malware and Trojans has further emphasized its importance in security-critical applications.

NFC, commonly used in payments, initially relied on expensive and inflexible hardware secure elements to store secrets. However, HCE enables cost-effective NFC transactions in a software environment without secure elements. Platforms like Android (from version 4.4 KitKat) adopted HCE, and payment networks such as VISA and MasterCard recommend white-box cryptography for securing tokens [16]. Memory leakage remains another major concern, caused by threats such as malware, Trojans, buffer overflows, and cold-boot attacks [23]. White-box security notions such as *Space-Hardness* (for one-time leakage) and *Longevity* (for continuous leakage) address these issues by assuming limited leakage channel capacity [7, 26]. White-box ciphers with high space-hardness and longevity effectively mitigate leakage risks. Space-hard ciphers prevent Trojans from exploiting low-capacity channels, while longevity ensures protection against gradual data leaks. Although several space-hard ciphers exist, none provide longevity. In fact, current ciphers are not only longevity-insecure against adaptive adversaries but also against non-adaptive adversaries. The existence of threats like Heartbleed [30], Cold Boot [23], and side-channel attacks makes this scenario more complicated, and a longevity secure cipher against such adversaries is highly desirable. Below, we highlight more details on the threats in a read-only, non-adaptive adversarial setting.

1.2 Threats in the Read-only Non-adaptive Adversarial Setting

For powerful cybersecurity threats, it is assumed that the adversary has full control over the execution environment, including the ability to read and write memory and execute code. However, with the rise of malware, read-only non-adaptive attacks have also become increasingly prevalent. Such malware, including that which utilizes memory dumps and memory-scanning techniques, gathers large amounts of sensitive information for identity theft or sale on the dark Web. In the following, we discuss a few of the prominent examples of such attacks.

- **Heartbleed Attack:** The Heartbleed vulnerability [30], discovered in 2014, exploited a critical flaw in the OpenSSL cryptographic library. This read-only attack stemmed from improper input validation in the "heartbeat" extension of the TLS protocol. By sending a crafted request, attackers could extract up to 64KB of a server's memory, exposing sensitive information like private keys, session cookies, and passwords, all without altering or corrupting the data. In addition to being read-only, Heartbleed is non-adaptive, as it does not depend on feedback or iterative refinement of queries. A single malicious request suffices

to extract a memory chunk, distinguishing it from adaptive attacks that adjust based on real-time responses.

- **Cold Boot Attack:** The Cold Boot Attack [23] is another notable example of a read-only and non-adaptive attack. It exploits residual data in volatile memory (e.g., RAM) to extract sensitive information such as encryption keys or passwords. Being read-only, it retrieves data without making any changes in the system's state. Additionally, it is non-adaptive, as attackers extract and analyze memory data offline without iterative queries or feedback. Once accessed, the memory data is dumped as-is, with no further interaction, distinguishing it from adaptive attacks.
- **Side Channel Attacks:** Side Channel attacks are passive read-only attacks because they passively observe physical phenomena without altering system behavior. In fact, they also exhibit a non-adaptive nature when they analyze pre-collected data without requiring real-time feedback or adjustments. This combination makes them stealthy and powerful, particularly against systems with inadequate protection against physical information leakage.

1.3 Existing Literature

Recognizing the challenges in secure software solutions, Chow *et al.* [12,13] introduced white-box cryptography with white-box implementations of DES (Data Encryption Standard) and AES (Advanced Encryption Standard), aiming for key-extraction resistance. Subsequent works introduced new security notions, such as protection against code leakage, along with supporting designs. Below, we briefly review the related works.

- **White-box Secure AES and DES:** Chow *et al.* [12,13] introduced the first white-box implementations of DES and AES, focusing on key-extraction resistance against white-box adversaries. Their method used look-up table networks masked with random linear input and non-linear output masking to conceal round key bytes. While later AES-based white-box designs aimed at key-extraction resistance [10,25,27,34], all were eventually broken [5,15,28,29,33].
- **Space-Hardness or Incompressibility under One-time Leakage:** Bogdanov *et al.* proposed SPACE [7], a white-box cipher resistant to key extraction, introducing a new white-box security notion denoted by *space-hardness*. This guarantees that even one-time partial table leakage doesn't aid in copying the encryption program. SPACE achieves high space-hardness by making code-leakage infeasible. Several other related works have been proposed [6–8,20] addressing similar security notions. Specifically, the proposal SPNbox [8] is designed to achieve high efficiency (in fact, currently the most efficient one).
- **Longevity or Incompressibility under Continuous Leakage:** While space-hard ciphers address one-time leakage, longevity tackles a stronger adversarial scenario of continuous leakage. Koike *et al.* [26] introduced the concept of *Longevity*, ensuring functionality despite gradual code leaks. The design is based on periodical code updates without changing the functionality, requiring adversaries to re-leak data from scratch. The authors in this paper proposed a white-box cipher denoted by Yoroi, claiming longevity security. However, Yoroi's longevity claims were later broken [32], leaving no current white-box cipher capable of achieving longevity under continuous leakage.
- **Leakage Resilient Cryptography :** [24] surveys on the leakage-resilient cryptography mainly in the public-key setting. Mainly, the research works mentioned in this paper assume that secrets are first encoded and then stored, and then leaked from storage. The papers [1,4,14,17,19] discuss about leakage from storage in detail. In addition, Davi *et al.*, in [14], introduced leakage resilient storage and proposed two constructions by splitting the stored encoded secret into two components, which are secure under bounded leakage. Another

design is the leakage-resilient public key encryption scheme [1], with the secret stored in two components communicated during the secret update, which is secure against continuous leakage. Here, the assumption is that the secret is stored in two components, and each of them leaks independently for the scheme. Next, Dodis *et al.* [17] proposed the construction which updates each component without communication. The leakage during the encoding process is considered in [19]. Note that all the above works deal with side channel leakages, where partial access to the system is available to the adversary through the leakages. Unlike side-channel models, white-box adversaries face no physical constraints. Though both white-box and side-channel attacks target implementations, white-box attacks are generally much stronger in software.

- **White-box Secure Even-Mansour (WEM):** WEM [11], based on Even-Mansour (EM), achieves a balance between encryption speed and space-hardness. Despite requiring 12 rounds for 112-bit space-hardness, two-round WEM is insecure under space-hardness. Extending WEM to a design with fewer rounds while maintaining high space-hardness and achieving longevity remains a significant challenge. Such improvements could provide a secure and efficient paradigm for white-box cryptography.

1.4 Motivations behind Our Work

Designing a white-box cipher with space-hardness and longevity is highly practical but challenging due to the adversary’s extreme power. The field is relatively young, lacking well-analyzed reference ciphers as base designs. Additionally, evolving non-black-box attacks make design approaches more heuristic, reducing security confidence. In fact, some existing designs have been proven insecure in the new attack models just after they are proposed (for example, Yoroi has been proven insecure under the longevity security notion). Hence, desirable designs should have the following characteristics.

1. It should be robust in security properties to counter various attacks. For example, achieving longevity alongside space-hardness is highly desirable.
2. Components must be securely integrated, as insufficient elements can compromise security.
3. Comprehensive security analysis is essential for confidence in the design. For instance, Yoroi lacked sufficient analysis of partial Sbox leakage, making it vulnerable to longevity attacks.

Achieving longevity is challenging due to the block cipher’s diffusion layer, which linearly mixes the state. However, we also observe that we can still achieve longevity if the adversary behaves in a non-adaptive way, if we use a chain of Sboxes. We point out that this chain of Sboxes

- does not have any impact on the black-box security,
- significantly uplifts white-box security in the non-adaptive setting, even with a lower number of rounds, and
- By inserting canceling encryption-decryption functions between two Sboxes in this chain (see Section 6.2), we can update the code by modifying these functions without altering functionality.

Hence, we choose a Substitution Permutation Network (SPN)-based block cipher with many rounds that leads to high latency when replacing the Sbox with a chain of Sboxes. We select the two-round Even-Mansour (EM) that offers high black-box security with two rounds. WEM, a white-box secure construction based on EM’s black-box security, is chosen as the base cipher. We extend WEM to EWEM (Extended WEM) instances where the keyed Sbox is replaced with a chain of Sboxes. We provide the first new and generic security analysis of EWEM with collision in the chain of Sboxes. This analysis can be applied to any existing SPN-based white-box ciphers. Furthermore, the chained structure allows inserting a canceling encryption-decryption

pair between any two Sboxes in this chain, whose effects cancel out. This enables periodic code updates by changing only the key of this canceling encryption-decryption pair to achieve longevity. Although EWEMrl and EWEMxl, which achieve longevity, are the main constructions of this work, we present EWEMr and EWEMx as the base design for the constructions EWEMrl and EWEMxl. Since EWEMr cannot achieve space-hardness under adaptive-chosen space setting, it is extended to EWEMx. Lastly, our instances use AES round functions due to the availability of the AES-NI instruction set.

1.5 Our Attack Model with the Non-adaptive Read-only Adversarial Setting

The white-box adversarial setting assumes two adversaries: (1) a local adversary (e.g., malware) that stealthily stays and leaks data from the system, and (2) a remote adversary that receives the leakage and aims to fulfill its security goal. In this paper, for the white-box security of EWEMrl, we assume a non-adaptive remote adversary (cannot request for leakages adaptively) and a read-only local adversary (can only read and exfiltrate data but performs no computation). The local attacker (e.g., malware),

- can have a complete view of the implementation code, but cannot update the code, can not compute anything in the system, and
- the malware can either make arbitrary code leakage (known space) or obtain non-adaptive queries (chosen space) and outputs to the remote adversary.

Hence, the remote adversary can either obtain arbitrary table input-output pairs from the malware or query the malware with a set of non-adaptively chosen inputs and obtain the corresponding outputs. Based on the already discussed threat actors (e.g., Heartbleed, Cold Boot Attacks), this model seems reasonable to consider in our attack.

1.5.1 Relevance of Non-adaptive Scenario for Longevity

Space-hardness refers to incompressibility under one-time leakage, where malware collects data and exfiltrates it all at once. This poses a risk of detection. On the other hand, longevity refers to incompressibility under continuous leakage, where malware gradually collects data and leaks a small amount of data continuously (e.g., 10 MB per day) to avoid detection. While continuous leakage can be seen as a sequence of one-time leakages, the key property of non-adaptivity can be applied in both settings, making both space hardness and longevity crucial considerations for security.

1.6 Our Contributions

In this work, we propose our longevity secure designs, EWEMrl and EWEMxl. To make our manuscript easy to read and well-structured, we start with our base designs EWEMr and EWEMx. EWEMr achieves high space-hardness bounds against non-adaptive adversaries but is insecure against an adaptive one. EWEMx achieves high space-hardness bounds even against adaptive adversaries, but is less efficient than EWEMr. Finally, we propose our longevity secure designs EWEMrl based on EWEMr and EWEMxl based on EWEMx, such that

- EWEMrl achieves longevity as well as space-hardness against non-adaptive adversaries,
- EWEMxl achieves longevity against non-adaptive adversaries, and space-hardness against adaptive adversaries, and
- EWEMrl is faster than EWEMxl.

Precisely, we present the following.

1. **Proposal of EWEMr:** We propose EWEMr, a white-box space-hard secure block cipher against non-adaptive, read-only malware. It generalizes WEM using ℓ -iteration of Sboxes, reducing random permutation calls via higher table look-ups. We introduce a new, generic space-hardness proof applicable to all SPN-based white-box ciphers. While not secure against adaptive adversaries or longevity-secure, EWEMr's structure supports longevity-secure designs. Details in Section 3 and 4.
2. **Proposal of EWEMx:** To prevent adaptively chosen space attacks, we sandwich one EWEMr call between two others, forming EWEMx (Fig.2), which achieves space-hardness and hybrid code-lifting security against adaptive adversaries, but not longevity. Details in Section 3 and 5.
3. **Proposal of EWEMrl:** Our main proposal, EWEMrl (Fig.8) extends EWEMr for longevity against non-adaptive read-only adversaries by replacing $\mathcal{S}^\ell$ with independently sampled secret Sboxes and periodic code updates via canceling encryption-decryption layers. This ensures previous leaks do not help, lowers table traversal probability, and applies against memory dumps, Heartbleed, Cold Boot, and side-channel attacks. Details in Section 6.
4. **Proposal of EWEMxl:** We briefly mention EWEMxl, a sandwich of EWEMrl between two EWEMr calls. It provides high space-hardness and security against hybrid code-lifting attacks under adaptive adversaries (one-time leakage) and longevity under continuous leakage. Outer EWEMr layers enhance space-hardness, while the central EWEMrl provides longevity. Details in Section 6.10.
5. **Software Implementations:** Both the Sbox and permutation layer efficiently use the AES-NI instruction set on Intel processors. We present competitive implementation results for EWEMr and EWEMrl instances. Details in Section 9.

2 Preliminaries

2.1 White-box Encryption

Definition 1 (Encryption Scheme). An encryption scheme is a mapping $E : \mathcal{K} \times \mathcal{R} \times \mathcal{P} \rightarrow \mathcal{C}$, that takes as input a key $K \in \mathcal{K}$, a plaintext $P \in \mathcal{P}$, and possibly some randomness $r \in \mathcal{R}$ and outputs a ciphertext $C \in \mathcal{C}$. Additionally, E is required to be invertible, such that there exists a decryption function $D : \mathcal{K} \times \mathcal{C} \rightarrow \mathcal{M}$, for which $D(K, E(K, r, P)) = P, \forall K, r, P$.

Definition 2 (White-box Compiler). A compiler Com for space-hard block ciphers is a function from $\mathcal{K} \times \mathcal{R}$ to $\mathcal{T}$ (a randomness could be optionally r drawn from a randomness space $\mathcal{R}$). Com outputs a table $T \in \mathcal{T}$. Then there exists a program $\tilde{E}_T$, functionally the same as E_K (i.e., $\tilde{E}_T(P) = E_K(P), \forall P \in \mathcal{P}$). We denote the size of the table (code) by $\sigma(T)$. Notably, $\sigma(T)$ is significantly larger than that of the original key K .

Definition 3 (White-box Encryption Scheme [20]). A white-box encryption scheme is a tuple of two encryption schemes E_1 and E_2 such that,

$$E_1 : \mathcal{K} \times \mathcal{P} \times \mathcal{R} \rightarrow \mathcal{C}, E_2 : \mathcal{T} \times \mathcal{P} \times \mathcal{R} \rightarrow \mathcal{C},$$

along with a white-box compiler $\text{Com} : \mathcal{K} \rightarrow \mathcal{T}$, such that $\forall K \in \mathcal{K}, E_1(K, \cdot, \cdot)$ is functionally equivalent to $E_2(T, \cdot, \cdot)$ where $T = \text{Com}(K)$.

From Definition 3, E_1 can be viewed as a traditional encryption scheme based on a master key $K \in \mathcal{K}$, and E_2 can be viewed as its white-box implementation version based on the table $T \leftarrow \text{Com}(K)$. To differentiate E_1 and E_2 , E_1 can be called the cipher and E_2 is the white-box implementation.

Table 1: Benchmarking of EWEM family of ciphers, in terms of space-hardness and longevity. ‘-’ indicates not applicable. $\sigma(T)$ is code (table) size; KS = Known Space, CS = Chosen Space, ACS = Adaptive Chosen Space, AS = Adaptive Setting, NAS = Non-Adaptive Setting. In “Space-hardness”, “ (c, Z) ” denotes that even if $\frac{1}{2^c}$ part of the table leaks, adversary’s success probability is $\leq 2^{-Z}$. In “Longevity Claim”, “ (c, Z) ” also denotes that even if $\frac{1}{2^c}$ part of the table leaks per update, adversary’s success probability is $\leq 2^{-Z}$. For the longevity claim of Yoro-16, the claim of (6, 128) is “Broken”, which means longevity is broken by an attacker without non-volatile memory. For Yoro-32, the longevity claim is broken in the standard Known Space model.

Cipher	Space-Hardness	Space-Hardness	Longevity Secure	Longevity	Security against
	KS/ CS	ACS	Yes/ No	Claim	Hybrid Code Lifting
EWEMr ¹⁶ [2, 8, 8, AES ₆]	(0.687, 128)	Not Secure	No	No	Secure in NAS
EWEMr ¹⁶ [2, 16, 8, AES ₆]	(0.355, 128)	Not Secure	No	No	Secure in NAS
EWEMx ¹⁶ [2, 2, 2, 1, 8, 1, 8, AES ₁ , AES ₆ , AES ₁]	(0.55, 128)	(0.55, 118.10)	No	No	Secure in both AS & NAS
EWEMx ¹⁶ [2, 2, 2, 1, 16, 1, 8, AES ₁ , AES ₁ , AES ₆ , AES ₁]	(0.3245, 128)	(0.3245, 117.76)	No	No	Secure in both AS & NAS
WEM [16(11, 1, 8, AES ₆)] [11]	(1.32, 128)	(2, 112)	No	No	-
SPNbox – 16 [8]	(1.61, 128)	(1.12, 88.4)	No	No	Secure in both AS & NAS
SPACE – 16 [7]	(1, 128)	(1.14, 119.72)	No	No	-
EWEMr ³² [2, 8, 4, AES ₆]	(1.334, 128)	Not Secure	No	No	Secure in NAS
EWEMr ³² [2, 16, 4, AES ₆]	(0.667, 128)	Not Secure	No	No	Secure in NAS
EWEMx ³² [2, 2, 2, 1, 8, 1, 4, AES ₁ , AES ₁ , AES ₆ , AES ₁]	(1.067, 128)	(1.067, 102.05)	No	No	Secure in both AS & NAS
EWEMx ³² [2, 2, 2, 1, 16, 1, 4, AES ₁ , AES ₁ , AES ₆ , AES ₁]	(0.593, 128)	(0.593, 101.55)	No	No	Secure in both AS & NAS
SPNbox – 32 [8]	(3.20, 128)	(3.20, 105.12)	No	No	Secure in both AS & NAS
SPACE – 32 [7]	(1, 128)	(1.14, 103.72)	No	No	-
Yoro – 16 [26]	(2, 128)	(2.04, 119.83)	No	(6, 128), Broken	Not Secure
EWEMr ¹⁶ [2, 12, 8, AES ₆ , b] b = 1010001110	(0.465, 128)	Not Secure	Yes	(6, 72)	Secure in NAS
EWEMr ¹⁶ [2, 21, 8, AES ₆ , b] b = 001011000011101001	(0.276, 128)	Not Secure	Yes	(5, 105)	Secure in NAS
EWEMr ¹⁶ [2, 27, 8, AES ₆ , b] b = 0000101100001111010010001	(0.221, 128)	Not Secure	Yes	(4, 108)	Secure in NAS
EWEMx ¹⁶ [2, 2, 2, 1, 12, 1, 8, AES ₁ , AES ₁ , AES ₆ , AES ₁ , b] b = 1010001110	(0.404, 128)	(0.404, 117.88)	Yes	(6, 72)	Secure in both AS & NAS
EWEMx ¹⁶ [2, 2, 2, 1, 21, 1, 8, AES ₁ , AES ₁ , AES ₆ , AES ₁ , b] b = 001011000011101001	(0.266, 128)	(0.266, 117.69)	Yes	(5, 105)	Secure in both AS & NAS
EWEMx ¹⁶ [2, 2, 2, 1, 27, 1, 8, AES ₁ , AES ₁ , AES ₆ , AES ₁ , b] b = 0000101100001111010010001	(0.225, 128)	(0.225, 117.66)	Yes	(4, 108)	Secure in both AS & NAS
Yoro – 32 [26]	(2, 128)	(2.04, 103.84)	No	(6, 128), Broken	Not Secure
EWEMr ³² [2, 16, 4, AES ₆ , b] b = 00000000000000	(0.667, 128)	Not Secure	Yes	(6, 96)	Secure in NAS
EWEMr ³² [2, 16, 4, AES ₆ , b] b = 00000011111100	(0.667, 128)	Not Secure	Yes	(6, 96)	Secure in NAS
EWEMr ³² [2, 18, 4, AES ₆ , b] b = 0000001111110000	(0.593, 128)	Not Secure	Yes	(6, 108)	Secure in NAS
EWEMx ³² [2, 2, 2, 1, 16, 1, 4, AES ₁ , AES ₁ , AES ₆ , AES ₁ , b] b = 00000000000000	(0.593, 128)	(0.593, 101.55)	Yes	(6, 96)	Secure in both AS & NAS
EWEMx ³² [2, 2, 2, 1, 16, 1, 4, AES ₁ , AES ₁ , AES ₆ , AES ₁ , b] b = 00000011111100	(0.593, 128)	(0.593, 101.55)	Yes	(6, 96)	Secure in both AS & NAS
EWEMx ³² [2, 2, 2, 1, 18, 1, 4, AES ₁ , AES ₁ , AES ₆ , AES ₁ , b] b = 0000001111110000	(0.534, 128)	(0.534, 101.50)	Yes	(6, 108)	Secure in both AS & NAS

2.2 Security in White-box

In a white-box setting, the adversary can extract the secret key or can lift the code of the implementation. The difficulty of code lifting attack is evaluated by space-hardness.

2.2.1 Space-Hardness

If we assume that the adversary can not extract the key given full access to $E_2(\text{Com}(K), \cdot, \cdot)$. As the adversary can not compute the ciphertext without K , alternatively, she can directly use $E_2(\text{Com}(K), \cdot, \cdot)$ as a larger key. This opens an avenue for a new attack, where the adversary lifts $E_2(\text{Com}(K), \cdot, \cdot)$ (instead of K) to another collaborative adversary. This is called a *Code-Lifting* attack. To address the code-lifting scenario, Bogdanov *et al.* [7], introduced the security notion denoted by *Space-Hardness*. We mention the definition directly from this paper.

Definition 4. ((M, Z) Space-Hardness) [7, 8]

An implementation of a block cipher E_K is (M, Z) space-hard if it is infeasible to encrypt (decrypt) any randomly chosen plaintext (ciphertext) with a probability of more than 2^{-Z} , given any code (table) of size less than M bits.

Intuitively, (M, Z) space-hardness provides an estimate on the leakage size M from the code, that can be isolated from the white-box environment to encrypt (or decrypt) any plaintext (or ciphertext) with probability more than 2^{-Z} . For the sake of simplicity, space-hardness bounds are denoted as (c, Z) instead of (M, Z) in Table 1 and Table 4, where (c, Z) represents that even if $\frac{1}{2^c}$ part of the table leaks, it is infeasible to encrypt (decrypt) a random plaintext (ciphertext) with probability more than 2^{-Z} . Here, we analyze the bounds of space hardness. Assuming table-based white-box implementations, we consider the following three adversarial models depending on the ability of the adversary as presented in ASIACRYPT 2016, by Bogdanov *et al.* [8].

1. **Known-Space (KS) Attack** : The adversary has the knowledge of q known input-output pairs from the tables. We symbolize this adversary as $\mathcal{A}_{KS}$.
2. **Chosen-Space (CS) Attack** : The adversary has the knowledge of q previously chosen input-output pairs from the tables. We symbolize this adversary as $\mathcal{A}_{CS}$.
3. **Adaptively-Chosen-Space (ACS) Attack** : The adversary obtains q adaptively chosen input-output pairs from the tables, which means she gets the output of an input before choosing the next input. We symbolize this adversary as $\mathcal{A}_{ACS}$.

2.2.2 Hybrid Code-Lifting Attack

The space-hardness intends to estimate the difficulty in encrypting (decrypting) a random plaintext (ciphertext) under one-time leakage against a white-box attacker. Space-hardness considers only the existence of a white-box attacker, but does not consider any black-box attacker receiving the code leakage. Todo *et al.* in [32], propose an attack setup denoted by *Hybrid Code-Lifting*. This attack model assumes two phases, where the first phase of the attack works in the white-box setting, and the second phase works in the black-box setting. The outcome of the attack (i.e., *win* or *loss*) is determined in the second phase. There are four parameters $(\lambda, \tau_{wbox}, q, \tau_{bbox})$ in this model, such that λ and τ_{wbox} are the parameters for the first phase, and q and τ_{bbox} are the parameters for the second phase. The definition is taken from [32].

Definition 5 ($(\lambda, \tau_{wbox}, q, \tau_{bbox})$ Hybrid Code-Lifting Attack). Let $E_T : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^n$ (this means the white-box code has table T) be the white-box implementation of an n -bit space-hard block cipher E_K . The attack against E_K consists of two phases.

- In the **first phase**, we assume the attacker who has full access to E_T . Precisely, the attacker can read all the table entries of T and perform arbitrary computations on it. The goal of the attacker is to lift at most λ bits from the implementation, with a time complexity upper bounded by $2^{\tau_{wbox}}$. The attacker may be assumed to be a malware or someone who steals the device temporarily.
- In the **second phase**, we assume an attacker who receives the leakage generated in the first phase. The attacker does not have white-box access to E_T , but only black-box access to E_K . Specifically, the attacker can only make black-box queries to the encryption/decryption oracles for E_K up to q times. The final goal of the adversary is to construct a function F , which is functionally equivalent to E_T . The time complexity of this phase is upper bounded by $2^{\tau_{bbox}}$.

Hybrid Code-Lifting in the Read-only Non-adaptive Setting: This is similar to the attack defined in Definition 5, except in the first phase, the attacker (malware) behaves as defined in Section 1.5. Precisely, if we recap from Section 1.5, the malware

- has a complete view of the implementation code but cannot update the code, and

- can make either arbitrary code leakage to its remote adversary or obtain non-adaptive queries (chosen space queries) from the remote adversary and return the outputs.

2.2.3 Longevity

Conventional space-hard block ciphers do not address continuous leakage. Koike *et al.* [26] introduced the notion of *longevity* in TCHES 2021, extending beyond traditional hardness. To mitigate continuous leakage, tables must be updated while preserving the block cipher’s functionality to prevent re-encrypting plaintexts. This ensures security against attackers who gradually leak small amounts of code over time to avoid detection (e.g., leaking 10 MB daily results in 1 GB leaked in 100 days). Regular table updates force adversaries to restart the leakage process from scratch after each update. Longevity is a stronger notion than space-hardness (which corresponds to one-time chunk of leakage). As an example, longevity lets the attacker eventually collect a large number of table entries by leaking a small amount of data/day for several days without getting detected. Longevity ensures that even in this scenario of periodic leakages, the cipher remains safe.

As the above example indicates, updating the code may prevent code-lifting under continuous leakage, but we need to re-encrypt (as updating the code changes the functionality) the data that is already encrypted by this cipher. This could be expensive as well as vulnerable (during the re-encryption). Longevity addresses the situation of updating the code without changing its functionality. In this context, the assumed adversarial model can specify a leakage function of bounded length. The adversary does not have access to the encryption-decryption oracles. The notion of Z -longevity is defined as follows. We directly use the definition proposed in [26]. Note that, in this case, Z signifies an upper bound on the number of leaked bits per session. In the definition of Z -longevity, the amount of leakage is not considered, as the code (table) is leaked continuously to the adversary. The amount of leakage comes into the scenario, after which the code (table) must be updated to prevent the attack against continuous leakage.

Definition 6 (Z -Longevity). A cryptographic cipher has Z -longevity if it is computationally hard to encrypt (or decrypt) randomly chosen plaintext (or ciphertext) with a probability of more than 2^{-Z} in the situation where the functionality remains constant, and code is continuously leaked to the adversary.

2.2.4 Reasons for Leaking Information in Table Entries

In a real scenario, the adversary can leak any information from the white-box implementation. As the total code (table) size is typically large (several GBs for the 32-bit version of the construction), it is hard for the adversary to leak the whole table. We are reasonably considering the leakage only in the number of table entries. In our construction, anything beyond the table is public. Hence, leaking anything outside the table does not provide any useful information.

2.3 EM and WEM

The EM scheme is a well-known approach for constructing a block cipher E from a public pseudo-random permutation $P : \{0, 1\}^n \mapsto \{0, 1\}^n$ and two n -bit keys K_0, K_1 . The Even-Mansour (EM) cipher $E : \{0, 1\}^{2n} \times \{0, 1\}^n \mapsto \{0, 1\}^n$ is defined as: $E_{K_0, K_1}(x) = P(x \oplus K_0) \oplus K_1$. The EM cipher has been studied intensively due to its simplicity. The structure obtained by iterating an EM scheme is called an Iterated Even-Mansour scheme and serves as an abstraction of many popular block cipher designs. Given r permutations $P_1, \dots, P_r : \{0, 1\}^n \mapsto \{0, 1\}^n$, and the secret keys $K = (K_0, K_1, \dots, K_r)$, the EM cipher is defined as follows:

$$E_K(X) = K_r \oplus P_r(P_{r-1}(\dots(P_1(K_0 \oplus X) \oplus K_1) \dots) \oplus K_{r-1}). \quad (1)$$

Algorithm 1 Subroutine $\chi_K^{\ell,w}$. Note that, $\text{parse}_w(X)$ divides X into w , n_{in} -bit words. Also, $K = K_0 \| K_1 \| \dots \| K_{\ell-1}$, such that $\forall i, |K_i| = \kappa$

Input: X

- 1: $X_0 \| X_1 \| \dots \| X_{w-1} \leftarrow \text{parse}_w(X)$
 - 2: **return** $(\mathcal{S}^\ell)_K(X_0) \| (\mathcal{S}^\ell)_K(X_1) \| \dots \| (\mathcal{S}^\ell)_K(X_{w-1})$, where $(\mathcal{S}^\ell)_K(X) = \mathcal{S}_{K_0} \circ \mathcal{S}_{K_1} \circ \dots \circ \mathcal{S}_{K_{\ell-1}}(X)$
-

WEM simply extends iterated EM design by replacing the key addition with keyed Sbox invocations. WEM has the structure $\chi_K^w \circ \pi \circ \chi_K^w \circ \dots \circ \pi \circ \chi_K^w$, where π is the underlying public random permutation, and χ_K^w is the parallel application of w keyed-secret Sboxes having the structure $\chi_K^w = \mathcal{S}_K(X_0) \| \mathcal{S}_K(X_1) \| \dots \| \mathcal{S}_K(X_{w-1})$.

Though WEM achieves $n - \log n$ -bit security in the black-box model for only $r = 2$, as it uses only one *iteration* of Sboxes in each round, the number of table lookups per round is small. Thus, a higher value of r is required to get a higher space-hardness bound.

3 Specification of EWEMr and EWEMx

In this section, we present a generic construction of the block cipher EWEMr and EWEMx. EWEMr and EWEMx are block ciphers with the block length $n = w \cdot n_{in}$ -bits with a κ -bit secret key. We begin by explaining the notations used to describe the design of EWEMr and EWEMx. Uppercase letters X, Y , etc., are used to represent various internal states of the cipher. Any state $X \in \mathbb{F}_2^n$ can be seen as an element of $X \in (\mathbb{F}_2^{n_{in}})^w$ where $w = n/n_{in}$ for a given n_{in} and n , i.e., an array of w words. The i -th word of the state X is denoted by X_i . The size of the word array is denoted by w . In terms of endianness, the leftmost (0-th) word is the most significant word. Throughout the paper, we use the term ℓ -iteration for Sboxes to denote ℓ -composition of Sboxes (precisely, to denote $\mathcal{S}^\ell = \underbrace{\mathcal{S} \circ \mathcal{S} \circ \dots \circ \mathcal{S}}_{\ell \text{ times}}$).

We will next describe the black-box and white-box constructions of EWEMr and EWEMx, and we have to construct them so that the functionality remains the same for both constructions.

3.1 Black-box Description of EWEMr

First, we would like to mention that we use the following notations.

- We use $(\mathcal{S}^\ell)_K$, such that $K = K_0 \| K_1 \| \dots \| K_{\ell-1}, \forall |K_i| = \kappa$ to denote $\mathcal{S}_{K_0} \circ \mathcal{S}_{K_1} \circ \dots \circ \mathcal{S}_{K_{\ell-1}}$.
- We use $\mathcal{S}_K^\ell$, such that $|K| = \kappa$ to denote $\underbrace{\mathcal{S}_K \circ \mathcal{S}_K \circ \dots \circ \mathcal{S}_K}_{\ell \text{ times}}$.

EWEMr extends WEM by replacing $\mathcal{S}_K$ with $(\mathcal{S}^\ell)_K$ for general ℓ , where $\ell = 1$ corresponds to WEM. Here, $\chi_K^{\ell,w}$ (described in Algorithm 1) is the keyed non-linear layer having the structure

$$\chi_K^{\ell,w} = (\mathcal{S}^\ell)_K(X_0) \| (\mathcal{S}^\ell)_K(X_1) \| \dots \| (\mathcal{S}^\ell)_K(X_{w-1}).$$

Therefore, EWEMr is a generic block cipher design paradigm that extends the design of WEM by congesting each Sbox with multi-iteration of Sboxes along with single-iteration of Sbox.

For better understanding, a schematic diagram of EWEMr for $r = 2, w = 8, \ell = 4$ and $\pi = AES_6$ is given in Fig.1, where AES_6 denotes 6 round AES with the key $\mathbf{0}^{128}$.

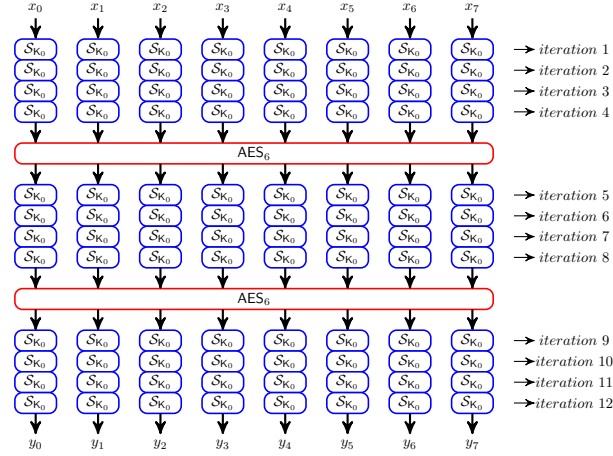


Figure 1: $\text{EWEMr}_{K_0}^{16}$ Black-box Encryption Algorithm for $r = 2, \ell = 4, w = 8, \pi = \text{AES}_6$. We also say that there are $3\ell = 12$ iterations of Sbox invocations. Note that we often use the term *iteration* as mentioned in this figure

Algorithm 2 $\text{EWEMr}_K^{n_{in}}[r, \ell, w, \pi]$ Black-box Description

- 1: Input: X
 - 2: **return** $\underbrace{\chi_K^{\ell, w} \circ \pi \circ \chi_K^{\ell, w} \circ \dots \circ \pi \circ \chi_K^{\ell, w}(X)}_{r\text{-times}}$
-

EWEMr is explicitly parametrized with the Sbox size n_{in} , number of rounds r , the number of Sbox *iterations* ℓ ¹, number of keyed Sboxes invocations w at each *iteration*, and the choice of π , and we denote it as $\text{EWEMr}^{n_{in}}[r, \ell, w, \pi]$. For the sake of simplicity, we often use $\text{EWEMr}^{n_{in}}[r, \ell, w, \pi](K, \cdot)$ and $\text{EWEMr}_K^{n_{in}}[r, \ell, w, \pi]$ interchangeably. It is also implicitly parameterized with the choice of S_K 's (for the sake of simplicity, S_K 's are not mentioned in the algorithm names).

3.1.1 Choice of Sbox

Let's review the general construction of S_K . S_K is a key-dependent bijective function that itself is a small SPN based block cipher with the round transformation of AES. It consists of $R_{n_{in}}$ rounds. S_K operates on a state $X = x_{t-1} \parallel \dots \parallel x_0$, where $t = n_{in}/8$ such that

$$S_K : (\mathbb{F}_{2^8})^t \rightarrow (\mathbb{F}_{2^8})^t, x \mapsto \left(\bigcirc_{i=1}^{R_{n_{in}}} (AK^i \circ MC_{n_{in}} \circ SB) \right) (AK^0(x)).$$

In this context, SB represents the simultaneous use of AES Sbox on each byte, and $MC_{n_{in}}$ represents the use of an MDS diffusion layer on ℓ bytes of the state. We use the following MDS matrices for $n_{in} = 32$ and $n_{in} = 16$:

$$MC_{32} = \begin{bmatrix} 02_x & 03_x & 01_x & 01_x \\ 01_x & 02_x & 03_x & 01_x \\ 01_x & 01_x & 02_x & 03_x \\ 03_x & 01_x & 01_x & 02_x \end{bmatrix} \quad MC_{16} = \begin{bmatrix} 02_x & 03_x \\ 01_x & 02_x \end{bmatrix}$$

¹It is important to emphasize the distinction between *round* and *iteration*. We call the outer rounds by *round*, and it is denoted by r (this is actually the number of public permutations). On the other hand, the number of keyed Sboxes used in each $(S^\ell)_K$ is called *iteration* and it is denoted by ℓ .

Algorithm 3 $\text{EWEMx}_{\mathbf{K}_1, \mathbf{K}_2, \mathbf{K}_3}^{n_{in}}[r_1, r_2, r_3, \ell_1, \ell_2, \ell_3, w, \pi, \pi_1, \pi_2, \pi_3]$ Black-box Description

Input: X
 1: **return** $\text{EWEMr}_{\mathbf{K}_3}^{n_{in}}[r_3, \ell_3, w, \pi_3] \circ \pi \circ \text{EWEMr}_{\mathbf{K}_2}^{n_{in}}[r_2, \ell_2, w, \pi_2] \circ \pi \circ \text{EWEMr}_{\mathbf{K}_1}^{n_{in}}[r_1, \ell_1, w, \pi_1](X)$

For $n_{in} = 16$ and $n_{in} = 32$, the number of rounds $R_{n_{in}}$ is 32 and 16, respectively. In both cases, a 4κ -bit key is embedded in $\mathcal{S}_\kappa$ to ensure security against Meet-in-the-Middle attacks as in [8]. This key can be derived by expanding the κ -bit master key K using any generic key derivation function KDF. For all concrete proposals, we propose to use SHAKE extendable output function [18].

Furthermore, in the case of EWEMrl, we use *maximum* four types of Sboxes based on a bit string b , all of them follow the same small SPN based block cipher as described above. However, note that here we need a total of 4 keys, each of size 4κ . Again, these also can be generated using any generic key derivation function KDF from the master key K . For all the concrete proposals, we propose to use SHAKE extendable output function [18]. Finally, for better understanding, we provide the description of $\chi_K^{\ell, w, b}$ for a given b in Algorithm 4.

3.1.2 Recommended Instances of EWEMr

For all of our recommended instances, we have $n = 128$ bit state and $\kappa = 512$ bit master key (each $\mathcal{S}_\kappa$ needs a 512-bit master key). We recommend the following concrete instances for EWEMr with $n_{in} = 32$ and $n_{in} = 16$, where AES_6 denotes 6 rounds of AES with some public parameter as the master key. We propose to use 0^{128} and 1^{128} for the first and second round master-key, respectively.² Our recommended instances are as follows (1) $\text{EWEMr}^{16}[2, 8, 8, \text{AES}_6]$, (2) $\text{EWEMr}^{16}[2, 16, 8, \text{AES}_6]$, (3) $\text{EWEMr}^{32}[2, 8, 4, \text{AES}_6]$, and (4) $\text{EWEMr}^{32}[2, 16, 4, \text{AES}_6]$.

Number of rounds: All of our proposed instances have $r = 2$ rounds, which is the minimum number of rounds required to provide security in black-box setting [11]. However, all the security analyses in this paper consider the general value of r .

The Choice of the Sbox: EWEMr utilizes a key-dependent nonlinear round (χ) that is applied to the entire state. This non-linear layer involves the parallel application of a key-dependent bijective function, namely $\mathcal{S}_\kappa^\ell$ to each word, where $\mathcal{S}_\kappa^\ell$ is the ℓ sequential applications of the key-dependent Sbox $\mathcal{S}_\kappa$ over n_{in} bit word as shown in Algorithm 1. EWEMr works for any incompressible keyed Sbox, $\mathcal{S}_\kappa$ which can be viewed as small block cipher. However, for the concrete instances, we fix the Sbox to be the small block cipher used in SPNbox [8]. We review the construction in Section 3.1.1.

The Choice of π : EWEMr works for any n -bit random permutation π . For our concrete proposals, we use 6 rounds of AES, denoted as AES_6 . This is directly adopted from the WEM construction. However, we differ in the sense of using a public key that is used as the master key of AES_6 . In the case of WEM, it was proposed to use a fixed key (0^{128}) for each application of AES. In the case of EWEMr we choose different public values for each application of AES_6 . The main motivation behind this is to prevent slide attacks, as discussed in [11]. Note that we use the same Sbox in each round of EWEMr, thus using the same public value in the AES_6 makes all the rounds symmetric. For our proposed constructions, we apply AES_6 for two times, thus we need two constants. We propose to use 0^{128} and 1^{128} .

²One can use any other public value as long as they are different for each round.

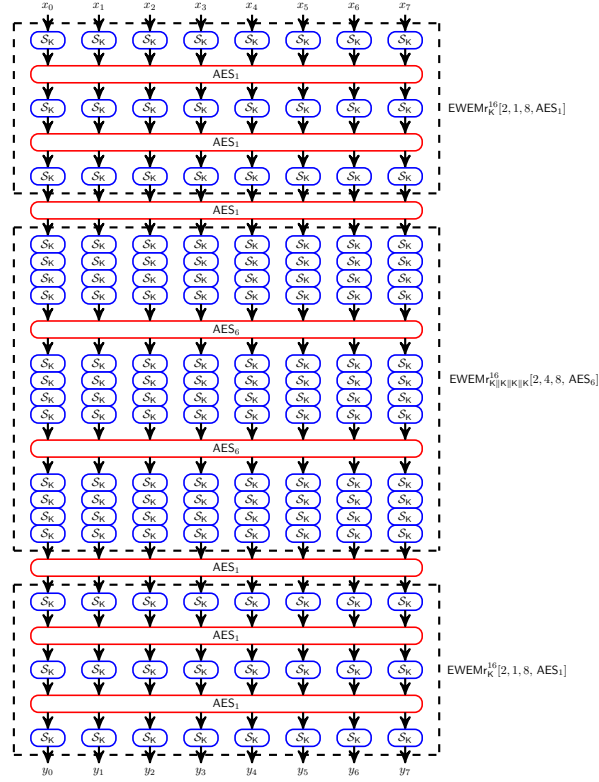


Figure 2: EWEMx^{16} Black-box Encryption Algorithm for $r_1 = 2, r_2 = 2, r_3 = 2, \ell_1 = 1, \ell_2 = 4, \ell_3 = 1, w = 8, \pi = \text{AES}_1, \pi_1 = \text{AES}_1, \pi_2 = \text{AES}_6, \pi_3 = \text{AES}_1$. Note that, here $\text{EWEMr}_{K||K||K}^{16}[2, 4, 8, \text{AES}_6]$ is sandwiched by $\text{EWEMr}_K^{16}[2, 1, 8, \text{AES}_1]$, and $\text{EWEMr}_K^{16}[2, 1, 8, \text{AES}_1]$

3.2 Black-box Description of EWEMx

EWEMx is described in Algorithm 3, which creates a sandwich over EWEMr , described in Fig.2. Precisely, in the design of EWEMx , one $\text{EWEMr}_{K_2}^{n_{in}}[r_2, \ell_2, w, \pi_2]$ is sandwiched by $\text{EWEMr}_{K_1}^{n_{in}}[r_1, \ell_1, w, \pi_1]$ and $\text{EWEMr}_{K_3}^{n_{in}}[r_3, \ell_3, w, \pi_3]$. EWEMx is parametrized with the number of rounds r_1 for the first outer part, r_2 for the middle part, r_3 for the last outer part, the number of *iterations* ℓ_1, ℓ_2 and ℓ_3 for the first outer part, the middle part and the last outer part respectively, Sbox size n_{in} , number of multi iteration keyed Sboxes invocations w , and the choice of the permutation π_1, π_2 and π_3 and we denote it as $\text{EWEMx}_{K_1, K_2, K_3}^{n_{in}}[r_1, r_2, r_3, \ell_1, \ell_2, \ell_3, w, \pi, \pi_1, \pi_2, \pi_3]$, described in Algorithm 3. It is also implicitly parameterized with the choice of S_K 's as EWEMr (for the sake of simplicity, S_K 's are not mentioned in the algorithm names).

We want to mention an important point. Our recommended instances for EWEMx contain a middle EWEMr call, where three Sbox layers are separated by two AES_6 layers. This alone provides us with sufficient black box security. Hence, for the upper and the lower EWEMr calls, we can use AES_1 , as we just need to linearly mix the Sbox outputs by an MDS matrix such that outputs from one Sbox alone in round i can not help us to compute the Sbox inputs in round $i + 1$ unless we know the other Sbox output values in round i .

3.3 White-box Description of EWEMr and EWEMx

To protect against *key-extraction* and *code-lifting attacks*, in the white-box implementation of EWEMr and EWEMx , we replace the operation S_K with a table look-up. Precisely, for a fixed key K , we replace S_K with a table $T_{n_{in}}$ (n_{in} emphasize that

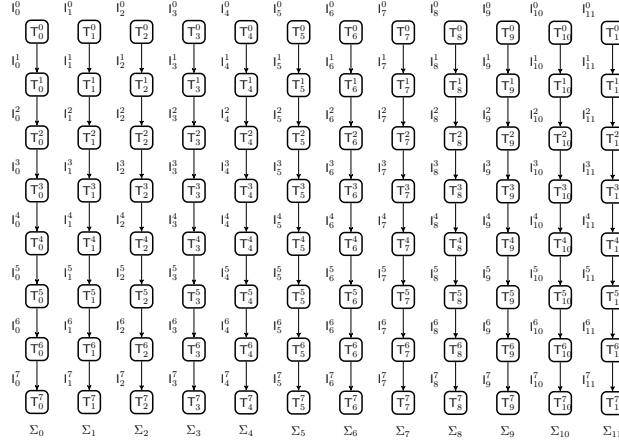


Figure 3: Variable assignments for $r = 2, w = 4$ and $\ell = 8$ represents Σ and four columns represent $\Sigma^{(\ell, w)}$ for $w = 4, \ell = 8$

$\mathcal{S}_K$ is a function over n_{in} bits and therefore the size of the table $T_{n_{in}}$ is $n_{in} \times 2^{n_{in}}$ bits). Furthermore, using the notation, $T_{n_{in}}^\ell$ denotes ℓ sequential application of $T_{n_{in}}$, i.e., $T_{n_{in}}^\ell : \mathbb{F}_2^{n_{in}} \rightarrow \mathbb{F}_2^{n_{in}}, x \mapsto T_{n_{in}}[T_{n_{in}}[\dots[T_{n_{in}}[x]]]]$. Furthermore, we denote the white-box implementation of $\chi_K^{\ell, w}$ as $\Sigma^{\ell, w}$. Note that $\Sigma^{\ell, w}$ can be viewed as a $\ell \times w$ matrix of $T_{n_{in}}$. Thus, often we refer to a sequence of ℓ applications of $T_{n_{in}}$ as a column and parallel applications of w many $T_{n_{in}}$ as a row.

Code (Table) Size: For both EWEMr and EWEMx, the code (table) size $\sigma(T_{n_{in}}) = n_{in} \cdot 2^{n_{in}}$ in bits.

4 White-box Security Analysis of EWEMr

In this section, we evaluate the key extraction resistance and space-hardness bounds of the construction EWEMr. Precisely, we evaluate the difficulty of code-lifting attacks by the notions of space-hardness [7], and also evaluate the security against recent advanced side-channel attacks [9, 21].

4.1 Key Extraction Security of EWEMr

In the white-box implementation of EWEMr, the full master keys are embedded inside the small table $T_{n_{in}}$ (representing only Sbox). This table is constructed from small block ciphers. Hence, key-extraction and decomposition attacks in the white-box setting are reduced to the key recovery problem of these small block ciphers in the black-box setting (presented in the black-box analysis of EWEMr).

Precisely, the table contains only the input-output pairs of $\mathcal{S}_K$ in the form of a constant array (for a fixed K). Hence, even if the adversary has full access to the code, she can only view the input-output pairs of $\mathcal{S}_K$. In fact, $\mathcal{S}_K$ is a small SPN-type block cipher with the key-recovery attack complexity of 2^{128} . They are based on the round transformation of the AES (Details of $\mathcal{S}_K$ are given in Sections 2.2 and 4.1 in [8]). Thus, observing one table entry provides no additional information beyond one black-box query to $\mathcal{S}_K$. Thus, the Key-extraction advantage (extraction of K given $E_2(\text{Com}(K, \cdot, \cdot))$, and denoted by $\text{Adv}_{\text{KeyExt-WB}}$ in the white-box model is upper-bounded by the key-recovery (for the underlying small block cipher $\mathcal{S}_K$) advantage in the black-box model, denoted by $\text{Adv}_{\text{KeyRec-BB}}$, i.e., $\text{Adv}_{\text{KeyExt-WB}} \leq \text{Adv}_{\text{KeyRec-BB}}$.

4.2 Space-Hardness of EWEMr under KS/CS

We now analyze the bound for weak (M, Z) space-hardness of EWEMr in the KS/CS attack setting and the basic definitions of space-hardness are in Section 2.2.1.

The evaluation of the space-hardness bound of EWEMr under known-space attacks is given in the following theorem. We would like to mention that, due to the randomly drawn plaintext, the inputs to the tables are unpredictable in advance, even in the chosen-space setting. Thus, the attack in the chosen space setting has no advantage over the attack in the known-space setting (see page 18, [8]), and we obtain the same bound as in Theorem 1. We evaluate the security of EWEMr in Theorem 1 modelling the underlying small block cipher $\mathcal{S}_K$ as a pseudorandom permutation.

Theorem 1. *For any adversary $\mathcal{A}_{KS}$ with a knowledge of a maximum of $\frac{2^{n_{in}}}{2^c}$ previously known input-output entries for $T_{n_{in}}$ (consists of $2^{n_{in}}$ entries), it is infeasible to encrypt a randomly chosen plaintext with $EWEMr^{n_{in}}[r, \ell, w, AES_6]$ with probability more than p such that*

$$p \leq \prod_{i=1}^{(r+1)w} ((i-1)(2\ell-1)2^{-n_{in}} + \max\{q^\ell, 2^{-n_{in}}\}),$$

where $q = 2^{-c}$ is the probability of computing one entry for a random input to the table with $\frac{1}{2^c}$ part of the table.

Proof: We show the upper bound of p by showing the $\mathcal{A}_{KS}$'s traversal through the columns of $\Sigma^{(\ell, w)}$ for a randomly chosen plaintext. In fact, in the r rounds there are total $(r+1)w$ columns ($r+1$ invocations of $\Sigma^{(\ell, w)}$ and w columns for each of the $\Sigma^{(\ell, w)}$). Let X^i denotes the input to the i -th $\Sigma^{(\ell, w)}$ for $i \in \{0, \dots, r\}$. Thus, the plaintext is $X^0 = X_0^0 \| X_1^0 \| \dots \| X_{w-1}^0$ and

$$X^1 = X_0^1 \| \dots \| X_{w-1}^1 = \pi_0 \circ \Sigma^{(\ell, w)}(X^0), \dots, X^r = X_0^r \| \dots \| X_{w-1}^r = \pi_{r-1} \circ \Sigma^{(\ell, w)}(X^{r-1}).$$

So the states X^i 's are placed as shown in equation 2.

$$X^0 \rightarrow \boxed{\Sigma^{f, w}} \rightarrow \boxed{\pi_0} \rightarrow X^1 \rightarrow \boxed{\Sigma^{f, w}} \rightarrow \boxed{\pi_1} \rightarrow \dots \rightarrow X^{r-1} \rightarrow \boxed{\Sigma^{f, w}} \rightarrow \boxed{\pi_{r-1}} \rightarrow X^r \rightarrow \boxed{\Sigma^{f, w}} \rightarrow \quad (2)$$

Since X^i 's are random states, we prove the upper bound of p , by finding the probability for $\mathcal{A}_{KS}$ to determine the output for random input $X^0 \| X^1 \| \dots \| X^r$ to the function $\Sigma = \Sigma^{(\ell, w)} \| \Sigma^{(\ell, w)} \| \dots \| \Sigma^{(\ell, w)}$. We represent the concatenated $X^0 \| X^1 \| \dots \| X^r$ by $I = I_0 \| I_1 \| \dots \| I_{(r+1)w-1} = X^0 \| X^1 \| \dots \| X^r$. Also, let us view Σ to be a concatenation of $(r+1)w$ columns such that

$$\Sigma = \Sigma^{(\ell, w)} \| \Sigma^{(\ell, w)} \| \dots \| \Sigma^{(\ell, w)} = \Sigma_0 \| \Sigma_1 \| \dots \| \Sigma_{(r+1)w-1},$$

where $\forall j, \Sigma_j$ computes $T_{n_{in}}^\ell[l_j]$. For the sake of simplicity, we describe the trail of tables $T_{n_{in}}^\ell$ corresponding to the j -th input word l_j as $T_j^{\ell-1} \circ \dots \circ T_j^1 \circ T_j^0$. Thus, we denote the input sequence to the corresponding tables as $l_j^0, l_j^1, \dots, l_j^{\ell-1}$, such that $l_j = l_j^0$. We can view the circuit Σ to have ℓ rows, and $(r+1)w$ columns such that l_j^i represents the table input for the i^{th} row and the j^{th} column, for $i \in \{0, 1, \dots, \ell-1\}, j \in \{0, 1, \dots, (r+1)w-1\}$. Therefore, the input to T_j^i is l_j^i . To be more comprehensive, we show the variable assignments in Fig.3 for an example of $r=2, w=4$ and $\ell=8$. Also note that, all the T_j^i s are **actually the same**, i.e., they all compute $T_{n_{in}}$.

We denote (i, j) , and (i', j') to be a collision pair, if $l_j^i = l_{j'}^{i'}$, and $(i, j) \neq (i', j')$, $i, i' \in \{0, \dots, \ell-1\}, j, j' \in \{0, \dots, (r+1)w-1\}$, and they are denoted as $((i, j), (i', j'))$. Note that $((i, j), (i', j'))$ and $((i', j'), (i, j))$ represent the same collision pair, and $((i, j), (i', j')) = ((i', j'), (i, j))$.

Importance of Collision. Analyzing collisions during table look-ups is very important to estimate the space-hardness bounds. If there is a collision in the input to T_j^i , and $T_{j'}^{i'}$, then all subsequent table look-ups in column j , and j' are the same. Hence, subsequent table look-ups in the two columns boil down to table look-ups in one column in the worst case (i.e., when $i = i' = 0$). This reduces the space-hardness bound. Thus, to estimate an upper bound, we need to analyze the collision possibilities.

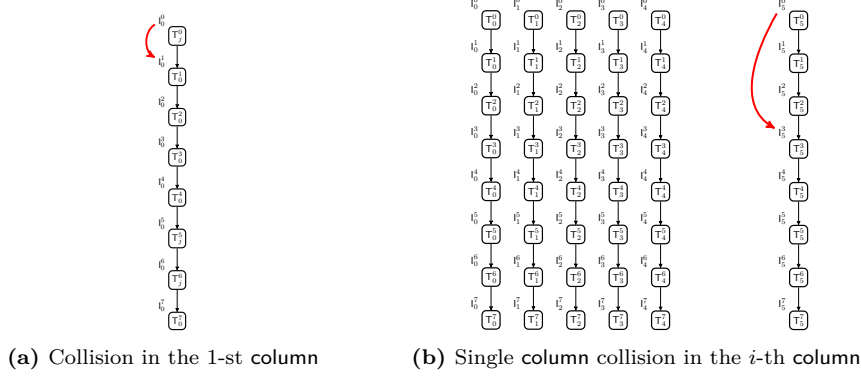


Figure 4: The i -th column has no collision with $\Sigma^{(i-1)}$. Here $\ell = 8$ and $i = 5$

In the following, we introduce two important properties of the collision pairs, namely *derived pair* and *equivalent pair*.

Definition 7. A collision pair $((i_1, j_1), (i_2, j_2))$ is said to derive another collision pair $((i_3, j_3), (i_4, j_4))$, if $i_{j_1}^{i_1} = i_{j_2}^{i_2}$ implies $i_{j_3}^{i_3} = i_{j_4}^{i_4}$. We also say that $((i_3, j_3), (i_4, j_4))$ is dependent on $((i_1, j_1), (i_2, j_2))$.

Definition 8. Two collision pairs $x = ((i_1, j_1), (i_2, j_2))$ and $y = ((i'_1, j'_1), (i'_2, j'_2))$ are equivalent, if x derives y and y derives x .

We now make the following useful observations for the collision pairs.

Observation 1. Collision pairs $((i_1, j_1), (i_2, j_2))$ and $((i_1 + h, j_1), (i_2 + h, j_2))$ are **equivalent** for any integer h (it is understood that both $i_1 + h$, and $i_2 + h$ do not exceed the value of ℓ).

Observation 2. Between any two columns $j \neq j'$, the following set

$$\mathbb{S} = \{((0, j), (i, j')) | i \in \{1, \dots, \ell - 1\}\} \cup \{((i, j), (0, j')) | i \in \{1, \dots, \ell - 1\}\} \cup \{((0, j), (0, j'))\}$$

covers all possible collisions between two columns, as in Fig. 5. This is because, from Observation 1,

- $((0, j), (i, j'))$ is equivalent to all possible collisions of the form $((h, j), (i + h, j'))$ (see Fig. 6b),
- $((i, j), (0, j'))$ is equivalent to all possible collisions of the form $((i + h, j), (h, j'))$ (see Fig. 6b), and
- $((0, j), (0, j'))$ is equivalent to all possible collisions of the form $((h, j), (h, j'))$ (see Fig. 6b).

Hence, between any two columns j and j' , there are a maximum $2\ell - 1$ possible independent collisions.

The value of q . We explicitly mention q in the theorem statement, for the following reason. q provides an upper bound for sequential table look-ups when the initial input is random and all the subsequent table inputs are distinct. We first discuss the probability of computing the output of h consecutive table look-ups (i.e.,

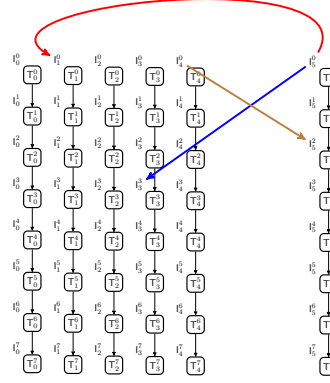
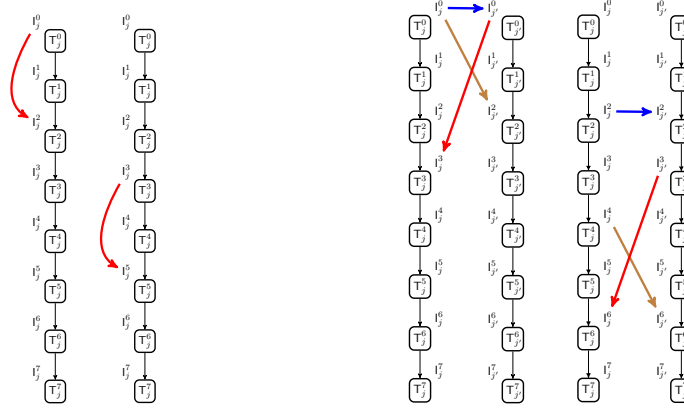


Figure 5: Different types collisions between two columns for $\ell = 8$



(a) Equivalent collisions in a single column (b) Equivalent collisions between two columns

Figure 6: Equivalent collisions for an example of $\ell = 8$. Same color arrows denote equivalent collisions

$\underbrace{T_{n_{in}} \circ T_{n_{in}} \circ \dots \circ T_{n_{in}}}_{h \text{ times}}$ given a random input and all subsequent table inputs are

distinct. Let the sequence of distinct look-up inputs are $X_0, X_1, \dots, X_{h-1}$. Start with X_0 and X_1 . Clearly, $X_0 \neq X_1$ (as they are distinct). To compute $T_{n_{in}}[X_1]$, there are two cases. The first case is, $(X_0, T_{n_{in}}[X_0])$ is among the $\frac{2^{n_{in}}}{2^c}$ table entries available to $\mathcal{A}_{KS}$ and this happens with probability $\frac{1}{2^c}$. The second case is, $(X_0, T_{n_{in}}[X_0])$ is not among the $\frac{2^{n_{in}}}{2^c}$ table entries and this happens with probability $1 - \frac{1}{2^c}$. We consider the two subcases. For the first subcase, X_1 can be looked up with probability $\frac{2^{n_{in}-c}-1}{2^{n_{in}-1}}$ (i.e., the event this entry is available to the adversary). For the second case, $\mathcal{A}_{KS}$ has to guess $T_{n_{in}}[X_1]$, and the guessing probability is very low (as finally we need to determine whether our guesses are correct or not from the final plaintext-ciphertext pair). We purposefully omit this term from the bound. This omission makes our bound negligibly loose but achieves a much simpler expression for the space-hardness bound. Hence, given that $\mathcal{A}_{KS}$ has $(X_0, T_{n_{in}}[X_0])$, the probability of determining $T_{n_{in}}[X_1]$ is $\frac{2^{n_{in}-c}-1}{2^{n_{in}-1}}$.

In a similar way, given that $\mathcal{A}_{KS}$ has $(X_0, T_{n_{in}}[X_0])$, and $(X_1, T_{n_{in}}[X_1])$, it determines $(X_2, T_{n_{in}}[X_2])$ with probability $\frac{2^{n_{in}-c}-2}{2^{n_{in}-2}}$. Given $\mathcal{A}_{KS}$ has $(X_0, T_{n_{in}}[X_0])$, $(X_1, T_{n_{in}}[X_1])$, $\dots$, $(X_{h-2}, T_{n_{in}}[X_{h-2}])$, the probability that $\mathcal{A}_{KS}$ determines $(X_{h-1}, T_{n_{in}}[X_{h-1}])$ is $\frac{2^{n_{in}-c}-h+1}{2^{n_{in}-h+1}}$. Now note that all the probabilities above are bounded by 2^{-c} . Hence,

the probability of traversing Σ_j for any j is q^ℓ , where $q = 2^{-c}$, and all the ℓ table look-ups are distinct. Precisely, the probability of determining $\Sigma_j(\mathbf{X})$, given a random $\mathbf{X}$, is upper bounded by q^ℓ .

Calculate p . p can be calculated by traversing through the columns of Σ . Let $\Sigma^{(i)}$ be the first i columns of Σ , i.e., $\Sigma^{(i)} = \Sigma_0 \| \Sigma_1 \| \dots \| \Sigma_{i-1}$. By $\text{Comp}(\Sigma^{(i)})$, we denote the probability that $\mathcal{A}_{KS}$ can compute the output $\mathbf{O}^{(i)} = \mathbf{O}_0 \| \mathbf{O}_1 \| \dots \| \mathbf{O}_{i-1}$ of $\Sigma^{(i)}$ given a random input $\mathbf{I}^i = \mathbf{I}_0 \| \mathbf{I}_1 \| \dots \| \mathbf{I}_{i-1}$. We first determine $\text{Comp}(\Sigma^{(1)})$, then $\text{Comp}(\Sigma^{(2)})$, until $\text{Comp}(\Sigma^{((r+1)w)})$. Precisely, in our proof, given $\text{Comp}(\Sigma^{(i-1)})$, we will find $\text{Comp}(\Sigma^{(i)})$. Note that, p is nothing but $\text{Comp}(\Sigma^{((r+1)w)})$.

Case 1: Compute $\text{Comp}(\Sigma^{(1)})$. For the first column, the column can either contain a collision or no collision.

- If there is no collision in this column then $\text{Comp}(\Sigma^{(1)})$ is q^ℓ (we have to traverse ℓ sequential table look-ups and each matching probability is upper bounded by q).
- If there is a collision in this column (as shown in Fig.4a), then $\text{Comp}(\Sigma^{(1)})$ is upper bounded by $q2^{-n_{in}}$. To understand this, we have to consider fixed points. When the collision is between $((0, 0), (1, 0))$, $\mathbf{I}_0^0$ is fixed point. Now, the probability that $\mathbf{I}_0^0$ is a fixed point is $2^{-n_{in}}$ (assuming the expected number of fixed points is 1), and the probability that $(\mathbf{I}_0^0, \mathbf{T}_0^0[\mathbf{I}_0^0])$ is available to the adversary and is bounded by q . All the subsequent $\ell - 1$ table look-ups can be done with probability one (i.e., $\mathbf{T}_0^1[\mathbf{I}_0] = \mathbf{I}_0$ implies $\mathbf{T}_j^{\ell-1} \circ \dots \circ \mathbf{T}_j^2 \circ \mathbf{T}_j^0[\mathbf{I}_0] = \mathbf{I}_0$). Therefore, the probability of determining $\mathbf{X}_0^1$ is upper bounded by $q2^{-n_{in}} < 2^{-n_{in}}$.

Considering both the above two cases, the probability of computing $\mathbf{O}_0$ is upper bounded by $\max\{q^\ell, 2^{-n_{in}}\}$.

Case i: Compute $\text{Comp}(\Sigma^{(i)})$. There are two cases: (1) The column i has no collision with $i - 1$ columns in $\Sigma^{(i-1)}$ (i.e., column i and $\Sigma^{(i-1)}$ are disconnected), and (2) there exists a collision between the column i and $\Sigma^{(i-1)}$ (i.e., there exists a collision between column i and column j , such that $j < i$).

Let Coll be the event (2), i.e., column i and $\Sigma^{(i-1)}$ are connected. Hence the probability of computing $\mathbf{O}_i$ from random $\mathbf{I}_i$ is

$$\begin{aligned} \Pr(\text{Compute } \mathbf{O}_i \text{ from } \mathbf{I}_i) &= \Pr(\text{Coll})\Pr(\text{Compute } \mathbf{O}_i \text{ from } \mathbf{I}_i | \text{Coll}) \\ &+ \Pr(\neg \text{Coll})\Pr(\text{Compute } \mathbf{O}_i \text{ from } \mathbf{I}_i | \neg \text{Coll}) \\ &\leq \Pr(\text{Coll}) + \Pr(\text{Compute } \mathbf{O}_i \text{ from } \mathbf{I}_i | \neg \text{Coll}). \end{aligned}$$

- **Compute $\Pr(\text{Coll})$.** To compute $\Pr(\text{Coll})$, there are total $(i - 1)(2\ell - 1)$ possible collisions between column i , and the $i - 1$ columns in $\Sigma^{(i-1)}$. This is because, for any column j among the first $i - 1$ columns, there are a total $2\ell - 1$ collisions between i , and j (various collisions are shown in Fig.5). Also, the probability of collision is $2^{-n_{in}}$. Hence, $\Pr(\text{Coll}) = (i - 1)(2\ell - 1)2^{-n_{in}}$.
- **Compute $\Pr(\text{Compute } \mathbf{O}_i \text{ from } \mathbf{I}_i | \neg \text{Coll})$.** When there is no collision between column i and $i - 1$ (as shown in Fig.4b) columns in $\Sigma^{(i-1)}$, $\mathbf{O}_i$ can be independently (independently from $\Sigma^{(i-1)}$) computed from $\mathbf{I}_i$. Thus, similar to Case 1,

$$\Pr(\text{Compute } \mathbf{O}_i \text{ from } \mathbf{I}_i | \neg \text{Coll}) \leq \max\{q^\ell, 2^{-n_{in}}\}.$$

Considering the above bounds, the probability is upper bounded by $(i - 1)(2\ell - 1)2^{-n_{in}} + \max\{q^\ell, 2^{-n_{in}}\}$. Thus, $\text{Comp}(\Sigma^{(i)}) = ((i - 1)(2\ell - 1)2^{-n_{in}} + \max\{q^\ell, 2^{-n_{in}}\})\text{Comp}(\Sigma^{(i-1)})$. After traversing all the columns, we compute

$$p \leq \text{Comp}(\Sigma^{((r+1)w)}) = \prod_{i=1}^{(r+1)w} ((i - 1)(2\ell - 1)2^{-n_{in}} + \max\{q^\ell, 2^{-n_{in}}\}).$$

□

4.2.1 Space-Hardness Bounds for EWEMr Instances under KS/CS

- EWEMr³²[2, 8, 4, AES₆] with $(2^{32}/2^{1.334}, 128)$ space-hardness,
- EWEMr¹⁶[2, 8, 8, AES₆] with $(2^{16}/2^{0.687}, 128)$ space-hardness.

For the sake of simplicity, we recommend only two instances of EWEMr with their KS/CS bounds. We provide the details of more instances in Table 1.

4.3 Space-Hardness of EWEMr under ACS

In this attack model, the remote adversary precomputes a list L of plaintext-ciphertext pairs by adaptive queries. After this phase, the adversary is given a challenge plaintext (or ciphertext) and needs to encrypt (or decrypt) it. If the challenge is in L , the adversary retrieves the corresponding ciphertext with probability one. If not, security depends on the amount of information leaked to the remote adversary non-adaptively. Due to adaptive access and sequential Sbox invocations to construct L , the adversary's success probability remains high. A local adversary (e.g., malware) can leak more. See Lemma 1, Theorem 2, and Section 4.4 for details. Hence, EWEMr instances are vulnerable under the ACS attack model.

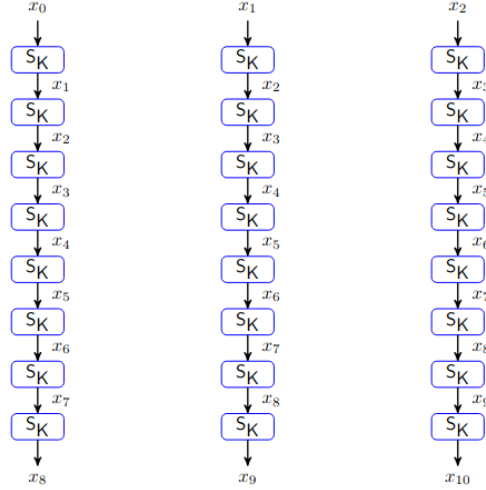


Figure 7: Three input-output pairs of SuperSbox with 10 adaptive queries to S_{box} for $\ell = 8$

4.4 Attack under ACS against EWEMr

In the ACS model, we can consider compositions of ℓ Sboxes as a **SuperSbox** (see Fig.7). The adversary can target a **SuperSbox** by making adaptive queries. We can obtain the maximum number of leaked entries of **SuperSboxes** from Lemma 1. In this case, by adaptive queries, we mean the remote adversary makes a sequence of Sbox (to table $T_{n_{in}}$ corresponds to S_K) queries to the local adversary. In this sequence, output for the i^{th} query is adaptively queried as the input for the $(i+1)^{th}$ query, for $1 \leq i \leq \ell - 1$. We can view the sequence $x_0, \dots, x_{\ell-2}, x_{\ell-1}$ as the remote adversary first queries x_0 to $T_{n_{in}}$ and obtains x_1 as the output. Next it makes x_1 as the query to $T_{n_{in}}$ and obtains x_2 , and so on such that finally $x_{\ell-2}$ is queried to $T_{n_{in}}$ and $x_{\ell-1}$ is the output. Hence $x_{\ell-1} = T_{n_{in}}^\ell(x_0)$ where $T_{n_{in}}^\ell$ corresponds to the SuperSbox S_K^ℓ . See the example in Fig.7.

Lemma 1. *If N table entries of the table $T_{n_{in}}$ are leaked adaptively, then maximum $(N - \ell + 1)$ entries can be obtained corresponding to the table $T_{n_{in}}^\ell$.*

Proof: To get the first input-output pair (x_0, x_ℓ) of the table $T_{n_{in}}^\ell$, we have to use ℓ entries $\{(x_0, x_1), (x_1, x_2), \dots, (x_{\ell-1}, x_\ell)\}$ of table $T_{n_{in}}$ adaptively. If we query x_ℓ to $T_{n_{in}}$ after the previous ℓ queries, then we obtain another one input-output pair of $T_{n_{in}}^\ell$ with only one additional entry of $T_{n_{in}}$. Similarly, we will get input-output pairs of $T_{n_{in}}^\ell$ one by one after every query to $T_{n_{in}}$ with the last output of $T_{n_{in}}$. Furthermore, the number of remaining table entries of $T_{n_{in}}$ is $N - \ell$ after getting the first input-output pair of $T_{n_{in}}^\ell$. Hence, we get $(N - \ell + 1)$ input-output pairs of $T_{n_{in}}^\ell$ from N entries of $T_{n_{in}}$. In Fig.7, the above process is described with a diagram for $N = 10, \ell = 8$. $\square$

Theorem 2. For any adversary $\mathcal{A}_{ACS}$ allowed up to $\frac{2^{n_{in}}}{2^c}$ number of adaptive table queries to $T_{n_{in}}$, it is infeasible to encrypt a randomly chosen plaintext with $EWEMr^{n_{in}}[r, \ell, w, AES_6]$ with probability more than $\left(\frac{2^{n_{in}} - (\ell - 1)}{2^{n_{in}}}\right)^{(r+1) \cdot w} \approx \left(\frac{1}{2^c}\right)^{(r+1) \cdot w}$.

The theorem can be proved in a straightforward way from Lemma 1. Hence, the attack complexity is low as it depends only on c but does not depend on ℓ . To prevent this attack, we can sandwich EWEMr to design EWEMx as described in Section 3.

4.5 Hybrid Code-Lifting Attack for EWEMr in the Read-only Non-adaptive Setting

We now apply the hybrid code-lifting attack against EWEMr, when the malware is read-only and non-adaptive (as mentioned in Section 1.5), and show that the construction of EWEMr can resist this attack. Note that, adaptive adversarial model could be considered when the number of rounds is r high. When r is small, the attack will work with low complexity. Hence, for general parameters, we consider non-adaptive, read-only adversaries. The idea of a hybrid code-lifting attack is described in Definition 5. On EWEMr, we apply a similar attack against SPNbox as described in [32].

4.5.1 Attack Idea

In the attack scenario, a white-box attacker leaks the implementation, and this leakage is used by a black-box adversary. If the target cipher is space-hard, it contains large amounts of information about the secret even after the leakage. For example, the code size for EWEMr is $2^{n_{in}} \cdot n_{in}$ bits. Assuming the quarter size of the code is leaked, the remaining $\frac{3 \cdot 2^{n_{in}} \cdot n_{in}}{4}$ bits is also huge compared to the usual block cipher implementations using 128 or 256-bit secret key for $n_{in} = 16, 32$. Thus, it is not easy for the black-box attacker to retrieve the full program using the leaked data.

To recover a huge amount of secret information (i.e., table entries), hybrid code-lifting aims to recover a part of (unleaked) table entries by the white-box adversary. The exact attack procedure depends on the target cipher. For example, the *guess-and-determine* approach has been used for SPNbox. We observe that EWEMr has a similar structure (except for the multi-iteration of Sboxes) and Sbox components are similar, providing security confidence, and the masking blockciphers do not leak any data about the Sbox interfaces. Hence, we apply the same technique on EWEMr, assuming a non-adaptive read-only adversary. As the only difference is ℓ -iteration Sboxes for EWEMr, the attacker has to use the tables ℓ -times consecutively for each round, and this makes the attack more complicated. We use the *Coupon Collector's* problem to analyze the cipher. In many cases, randomly picked table entries are recovered. Precisely, the attack often reveals already known table entries. This observation is closely related to the coupon collector's problem, such that the goal is to collect all table entries acting as coupons. Below, we first mention the theorem for the coupon collector's problem (we use the exact representation of this theorem from [32]), and then proceed with the attack on EWEMr.

Theorem 3 (Coupon Collector's Problem). *Let there be n coupons. Assume we already have k_0 coupons. When one trial randomly opens one coupon, the expected*

number of trials to collect all n coupons is $nH_{n-k_0} \approx n(\ln(n-k_0) + \gamma)$, where H_n is the n^{th} harmonic number, and γ is the Euler's constant, i.e., $\gamma \approx 0.577$.

The lemma (proofs are available in [32]) mentioned below follows from this theorem.

Lemma 2. *Let there be n coupons. Assume we already have k_0 coupons. When one trial randomly opens one coupon, the expected total number of coupons collected by t trials is*

$$k + (t - n(H_{n-k_0} - H_{n-k})) \cdot \frac{n-k}{n}, \text{ where } k = \lfloor n - e^{\ln(n-k_0) - \frac{t}{n}} \rfloor$$

4.6 Hybrid Code-Lifting Attack for EWEMr

Let us assume that total $\sigma(\mathbb{T}_{n_{in}})/4$ amount of codes will be lifted by the white-box adversary, where $\sigma(\mathbb{T}_{n_{in}})$ is the total code size (i.e., $\sigma(\mathbb{T}_{n_{in}}) = n_{in} \cdot 2^{n_{in}}$ bits). We also assume $r = 2$ and $\ell = 8$ in $\text{EWEMr}^{n_{in}}[r, \ell, w, AES_6]$, for the sake of simplicity. However, a similar proof will work for general r and ℓ . The attack on EWEMr has two phases.

1. **White-box Phase:** Note that, to encrypt a plaintext $(r+1)\ell w = 3\ell w$ table lookups are needed for $\mathbb{T}_{n_{in}}$, with read-only malware. The white-box attacker first chooses A_0 table entries from $\mathbb{T}_{n_{in}}$, and then, $\mathcal{N} \leq A_0^w$ plaintexts can be encrypted up to the output of the *iteration 1*. The attacker next extracts $(3\ell-2)w\mathcal{N}$ table entries to encrypt these $\mathcal{N}$ plaintexts up to the second last *iteration* (i.e., the *iteration* $3\ell-1$). Then $(3\ell-2)w\mathcal{N}$ (overlapping) table entries are extracted for $\mathbb{T}_{n_{in}}$. This can be mapped to the coupon collector's problem, where there are $2^{n_{in}}$ coupons for $\mathbb{T}_{n_{in}}$ and $2^{n_{in}-2}$ coupons are collected with $(3\ell-2)w\mathcal{N}$ trials. From Lemma 2, we get $2^{n_{in}-2} \approx 2^{n_{in}} - e^{\ln(2^{n_{in}}) - \frac{(3\ell-2)w\mathcal{N}}{2^{n_{in}}}}$. Thus, we get $\mathcal{N} \approx \lfloor \frac{2^{n_{in}} \times (\ln(2^{n_{in}}) - \ln(2^{n_{in}-2}) - \frac{w\mathcal{N}}{2^{n_{in}}})}{(3\ell-2)w} \rfloor$. Hence, $\mathcal{N} \approx 14040739$, $A_0 \approx 62$, for $n_{in} = 32$ and $\mathcal{N} \approx 107$, $A_0 \approx 2$, for $n_{in} = 16$, and $\ell = 8$, for both n_{in} . Note that these values of $\mathcal{N}$, $(3\ell-2)w\mathcal{N}$, trials will reveal $2^{n_{in}-2} - A_0$ table entries for $\mathbb{T}_{n_{in}}$, as $\mathbb{T}_{n_{in}}$ already chosen A_0 table entries.

2. **Black-box Phase:**

- (a) **First Step:** In this step, $\mathcal{N}$ chosen plaintexts are encrypted up to the invocation of the second last *iteration* (note that the attacker has already computed up to the second last *iteration* in the white-box phase). In this case, the attacker also queries the corresponding chosen plaintexts to the encryption oracle. Then, $w\mathcal{N}$ (overlapping) $\mathbb{T}_{n_{in}}$ entries are extracted from the last round. This can be mapped to the coupon collector's problem, so that there are $2^{n_{in}}$ coupons and $2^{n_{in}-2}$ coupons have already been collected, and some additional coupons are collected by the $(3\ell-2)w\mathcal{N}$ trials. By Lemma 2, the number of finally collected table entries from the table is estimated as $A \approx \lfloor 2^{n_{in}} - e^{\ln(2^{n_{in}} - 2^{n_{in}-2}) - \frac{w\mathcal{N}}{2^{n_{in}}}} \rfloor$. Hence, $A \approx 1115589833$, for $n_{in} = 32$, and $A \approx 17021$, for $n_{in} = 16$.
- (b) **Second Step:** This step aims to recover both tables fully. We have A table entries. By Theorem 3, we estimate the number of table extractions required to collect all the table entries. The number of trials is $t = 2^{n_{in}}(\ln(2^{n_{in}} - A) + 0.577)$, which is $2^{36.49}$, for $n_{in} = 32$ and $2^{19.51}$, for $n_{in} = 16$.

We check whether we can obtain these (overlapping) table entries. There are A^w chosen plaintexts that are encrypted up to the invocation of $\mathbb{T}_{n_{in}}$ in the first *iteration*. Each lookup for the table is known with probability $\frac{A}{2^{n_{in}}}$. Hence, the probability of encrypting these A^w chosen plaintexts up to the invocation of $\mathbb{T}_{n_{in}}$ in the second last *iteration* is $p = \left(\frac{A}{2^{n_{in}}}\right)^{(3\ell-2)w}$.

Now, $\text{check} = p \cdot A^w < 1$ for both $n_{in} = 32$ and 16. Hence, the above method cannot find any table entry from EWEMr^{32} and EWEMr^{16} .

Hence, as in the case of SPNbox-16 , we follow the same strategy *filter-then-guess*, for $\text{EWEMr}^{n_{in}}$. Step 1 (white-box phase) and Step 2(a) (black-box phase, first step) will be the same as discussed above. In Step 2(b), we use the *filter-then-guess* technique instead of the previous technique. We first find the value of $\mathcal{N}$, A_0 , and A . Following the same strategy, we find that $\mathcal{N} = 14040739$, $A_0 = 62$, $A = 1115589832$, for $n_{in} = 32$ and $\mathcal{N} = 107$, $A_0 = 2$, $A = 17021$, for $n_{in} = 16$.

In the case of u unknown table entries of $\mathbb{T}_{n_{in}}$ while encrypting a plaintext, we need to guess u unknown table entries of $\mathbb{T}_{n_{in}}$. There are two difficulties.

1. Is it possible to maintain a time complexity smaller than 2^{128} under the fact that we guess u unknown table entries of $\mathbb{T}_{n_{in}}$?
2. Is it possible to remove all the wrong guesses?

The filter-then-guess procedure is useful to address the first question. The attack first filters available data, reduces the size, and then guesses unknown table entries. Regarding the second question, the attack uses the scenario that some table entries are known in the last round, such that the wrong key can be filtered out by checking between known entries and ciphertexts from the encryption oracle.

Note that, there are a total of $3w\ell$ Sbox invocations (with w invocations at each of the 3ℓ iterations, for example, see Fig 1, which has $\ell = 4$), and we look-up $\underbrace{\mathbb{T}_{n_{in}} \parallel \mathbb{T}_{n_{in}} \parallel \dots \parallel \mathbb{T}_{n_{in}}}_{w \text{ times}}$ at each of the 3ℓ iterations. Let $\vec{u} = (u_1, u_2, \dots, u_{3\ell})$ be a

vector such that u_r is the number of the unknown table entry access in the r^{th} iteration. Note that we now have A^w chosen plaintexts, and we can encrypt them up to the first iteration. Hence $u_1 = 0$, and there are A^w entries for the invocation of the table in the second iteration. Given A^w inputs, u_2 accesses to unknown table entries are accepted. Therefore, the number of inputs to the second iteration generated by the filter-then-guess procedure is

$$A^w \times \left\{ \binom{w}{u_2} \times \left(1 - \frac{A}{2^{n_{in}}}\right)^{u_2} \times \left(\frac{A}{2^{n_{in}}}\right)^{(w-u_2)} \right\} \times (2^{n_{in}} - A)^{u_2}.$$

These are the inputs of the third iteration. Note that the time complexity is exactly the same as the number of chosen plaintexts. Given $\vec{u}$, the time complexity can be evaluated by the iterative applications, and we use only those $\vec{u}$ such that the time complexity is lower than 2^{128} . Consider $\mathbb{S}$ to be the set of such $\vec{u}$ tuples, and the total time complexity is the sum of the complexity for each $\vec{u} \in \mathbb{S}$. The probability that plaintexts can be correctly encrypted up to the second last iteration of $\mathbb{T}_{n_{in}}$ in the last round is

$$p = \sum_{\vec{u} \in \mathbb{S}} \left(\prod_{i=2}^{3\ell-1} \binom{w}{u_i} \right) \times \left(1 - \frac{A}{2^{n_{in}}}\right)^u \times \left(\frac{A}{2^{n_{in}}}\right)^{(3\ell-2) \times w - u},$$

where $u = \sum_{j=2}^{(3\ell-1)} u_j$. Hence, we get the following results.

- For $u = 3, n_{in} = 32$, we have $p = 2^{-149.94}$. Hence, $A^4 \times p = 2^{-29.72}$, i.e., no plaintext can be correctly encrypted up to the second last iteration, though all the guesses are correct.
- For $u = 4, n_{in} = 16$, we have $p = 2^{-313.58}$. Hence, $A^8 \times p = 2^{-201.14}$, i.e., no plaintext can be correctly encrypted up to the second last iteration, though all the guesses are correct.

Thus, for $u = 3$ and $u = 4$, no plaintext can be correctly encrypted up to the second last iteration, though all the guesses are correct. Hence, EWEMr is secure against hybrid code lifting attacks.

5 White-box Security Analysis of EWEMx

The key extraction security analysis for EWEMx is exactly the same as that of EWEMr. In this section, we evaluate the space-hardness bounds for EWEMx. We also evaluate the resistance of EWEMx against the hybrid code-lifting attack.

5.1 Space-Hardness of EWEMx under KS/ CS

We analyze weak (M, Z) space-hardness bound of EWEMx in the KS/CS setting. The evaluation of the space-hardness bound of EWEMx under the KS setting is given in the following theorem. The bound is the same in the CS setting.

Theorem 4. *For any adversary $\mathcal{A}_{KS}$ with a knowledge of a maximum of $\frac{2^{n_{in}}}{2^c}$ previously known table entries of $T_{n_{in}}$, it is infeasible to encrypt a randomly chosen plaintext with $\text{EWEMx}^{n_{in}}[r_1, r_2, r_3, \ell_1, \ell_2, \ell_3, w, \pi, \pi_1, \pi_2, \pi_3]$ with probability more than $p_1 = p'_1 p''_1 p'''_1$ with*

$$\begin{aligned} \bullet \quad p'_1 &= \prod_{i_1=1}^{(r_1+1)w} ((i_1 - 1)(2\ell_1 - 1)2^{-n_{in}} + \max\{q^{\ell_1}, 2^{-n_{in}}\}), \\ \bullet \quad p''_1 &= \prod_{i_2=1}^{(r_2+1)w} ((r_1+1)w(\ell_1+\ell_2-1)2^{-n_{in}} + (i_2-1)(2\ell_2-1)2^{-n_{in}} + \max\{q^{\ell_2}, 2^{-n_{in}}\}), \\ \bullet \quad p'''_1 &= \prod_{i_3=1}^{(r_3+1)w} ((r_1+1)w(\ell_1+\ell_3-1)2^{-n_{in}} + (r_2+1)w(\ell_2+\ell_3-1)2^{-n_{in}} + \\ &\quad (i_3-1)(2\ell_3-1)2^{-n_{in}} + \max\{q^{\ell_3}, 2^{-n_{in}}\}), \end{aligned}$$

such that $q = 2^{-c}$.

Proof: We use the same strategy as the proof of Theorem 1 to calculate p_1 for general $r_1, r_2, r_3, \ell_1, \ell_2$, and ℓ_3 . From $\text{EWEMr}_{K_3}^{n_{in}}[r_3, \ell_3, w, \pi_3] \circ \pi \circ \text{EWEMr}_{K_2}^{n_{in}}[r_2, \ell_2, w, \pi_2] \circ \pi \circ \text{EWEMr}_{K_1}^{n_{in}}[r_1, \ell_1, w, \pi_1]$ described in Algorithm 3, and similar to Fig.3, we now have a total of $(r_1+1)w + (r_2+1)w + (r_3+1)w = (r_1+r_2+r_3+3)w$ columns, such that

- all the first $(r_1+1)w$ columns consist of a chain of ℓ_1 Sboxes (for $\text{EWEMr}_{K_1}^{n_{in}}[r_1, \ell_1, w, \pi_1]$),
- all the next $(r_2+1)w$ columns consist of a chain of ℓ_2 Sboxes (for $\text{EWEMr}_{K_2}^{n_{in}}[r_2, \ell_2, w, \pi_2]$),
- all the last $(r_3+1)w$ columns consist of a chain of ℓ_3 Sboxes (for $\text{EWEMr}_{K_3}^{n_{in}}[r_3, \ell_3, w, \pi_3]$).

We now calculate the number of possible collisions between two columns, such that one column has a chain of ℓ_i Sboxes and the other has a chain of ℓ_j Sboxes with $i \neq j$. Following the same strategy in Observation 2, we can say that these two columns have a maximum $\ell_i + \ell_j - 1$ independent collisions. We now briefly describe p'_1, p''_1 , and p'''_1 . p'_1 is simply the probability "p" described in Theorem 1 for the $\text{EWEMr}_{K_1}^{n_{in}}[r_1, \ell_1, w, \pi_1]$ part, p''_1 is the same probability corresponding to $\text{EWEMr}_{K_2}^{n_{in}}[r_2, \ell_2, w, \pi_2]$ given the chains corresponding to $\text{EWEMr}_{K_1}^{n_{in}}[r_1, \ell_1, w, \pi_1]$. Similarly, p'''_1 is the same probability corresponding to $\text{EWEMr}_{K_3}^{n_{in}}[r_3, \ell_3, w, \pi_3]$ given the chains corresponding to $\text{EWEMr}_{K_1}^{n_{in}}[r_1, \ell_1, w, \pi_1]$ and $\text{EWEMr}_{K_2}^{n_{in}}[r_2, \ell_2, w, \pi_2]$. Hence, following the same strategy as probability for chain calculation in Theorem 1, we now infer that,

$$\begin{aligned} \bullet \quad p'_1 &= \prod_{i_1=1}^{(r_1+1)w} ((i_1 - 1)(2\ell_1 - 1)2^{-n_{in}} + \max\{q^{\ell_1}, 2^{-n_{in}}\}), \\ \bullet \quad p''_1 &= \prod_{i_2=1}^{(r_2+1)w} ((r_1+1)w(\ell_1+\ell_2-1)2^{-n_{in}} + (i_2-1)(2\ell_2-1)2^{-n_{in}} + \max\{q^{\ell_2}, 2^{-n_{in}}\}), \\ \bullet \quad p'''_1 &= \prod_{i_3=1}^{(r_3+1)w} ((r_1+1)w(\ell_1+\ell_3-1)2^{-n_{in}} + (r_2+1)w(\ell_2+\ell_3-1)2^{-n_{in}} + \\ &\quad (i_3-1)(2\ell_3-1)2^{-n_{in}} + \max\{q^{\ell_3}, 2^{-n_{in}}\}). \end{aligned}$$

□

5.1.1 Space-Hardness Bounds for EWEMx Instances under KS/CS

- $\text{EWEMx}^{32}[2, 2, 2, 1, 8, 1, 4, \text{AES}_1, \text{AES}_1, \text{AES}_6, \text{AES}_1]$: $(\frac{2^{32}}{2^{1.067}}, 128)$ space-hardness,
- $\text{EWEMx}^{16}[2, 2, 2, 1, 8, 1, 8, \text{AES}_1, \text{AES}_1, \text{AES}_6, \text{AES}_1]$: $(\frac{2^{16}}{2^{0.355}}, 128)$ space-hardness.

For the sake of simplicity, we recommend only two instances of EWEMx with their KS/CS bounds. We provide the details of more instances in Table 1.

5.2 Space-Hardness of EWEMx under ACS

Before discussing the theorem for ACS, we consider that the adversary can compute some input-output pairs corresponding to the table look-ups $T_{n_{in}}^\ell$ with adaptive queries to the table look-up $T_{n_{in}}$ by considering $T_{n_{in}}^\ell$ (where $T_{n_{in}}^\ell = \underbrace{T_{n_{in}} \circ T_{n_{in}} \circ \dots \circ T_{n_{in}}}_{\ell \text{ times}}$) as a single table look-up. We now mention Lemma 3 (directly taken from [8]), and prove Theorem 5.

Lemma 3. *For m table look-ups, the expected value of the number of used entries in the table is estimated as $(1 - (1 - \frac{1}{2^{n_{in}}})^m) \cdot 2^{n_{in}}$.*

Theorem 5. *For any adversary $\mathcal{A}_{ACS}$ allowed up to $\frac{2^{n_{in}}}{2^c}$ many adaptive queries to $T_{n_{in}}$, it is infeasible to encrypt a random plaintext using $\text{EWEMx}^{n_{in}}[r_1, r_2, r_3, \ell_1, \ell_2, \ell_3, w, \pi, \pi_1, \pi_2, \pi_3]$ with probability more than $A \cdot 2^{-128} + p_1$, where $A = \lceil \frac{\log_{e_{n_{in}}}(1-2^{-c})}{V} \rceil$, $e_{n_{in}} = 1 - \frac{1}{2^{n_{in}}}$, $V = w(r_1 + 1) + w(r_2 + 1) + w(r_3 + 1) + (\ell_1 + \ell_2 + \ell_3 - 3)$, $p_1 = p'_1 p''_1 p'''_1$ with $q = 2^{-c}$ and*

- $p'_1 = \prod_{i_1=1}^{(r_1+1)w} ((i_1 - 1)(2\ell_1 - 1)2^{-n_{in}} + \max\{q^{\ell_1}, 2^{-n_{in}}\})$,
- $p''_1 = \prod_{i_2=1}^{(r_2+1)w} ((r_1+1)w(\ell_1+\ell_2-1)2^{-n_{in}} + (i_2-1)(2\ell_2-1)2^{-n_{in}} + \max\{q^{\ell_2}, 2^{-n_{in}}\})$,
- $p'''_1 = \prod_{i_3=1}^{(r_3+1)w} ((r_1+1)w(\ell_1+\ell_3-1)2^{-n_{in}} + (r_2+1)w(\ell_2+\ell_3-1)2^{-n_{in}} + (i_3-1)(2\ell_3-1)2^{-n_{in}} + \max\{q^{\ell_3}, 2^{-n_{in}}\})$,

Proof: We follow the same proof strategy as Theorem 2 in [8]. From Lemma 1, we get a lower bound on the number of Sbox evaluations per encryption. Note that, one encryption makes a total of $(r_1 + 1)w$ invocations of the **SuperSbox** $S_K^{\ell_1}$, $(r_2 + 1)w$ invocations of the **SuperSbox** $S_K^{\ell_2}$, and $(r_3 + 1)w$ invocations of the **SuperSbox** $S_K^{\ell_3}$. From Lemma 1, we get that N evaluation of S_K^ℓ needs at least $N + \ell - 1$ entries need to be evaluated. This essentially gives a lower bound on the number of **SuperSbox** evaluations per encryption, which is $w(r_1+1)+\ell_1-1+w(r_2+1)+\ell_2-1+w(r_3+1)+\ell_3-1 = V$. Let A be the number of plaintext-ciphertext pairs computed by $\mathcal{A}_{ACS}$ using the adaptively leaked table entries with V **SuperSbox** evaluations per encryption. Now, from Lemma 3, $(1 - e_{n_{in}}^{AV})2^{n_{in}} \cdot n_{in}$ bits of space are needed to store the adaptively leaked entries from the table to compute A plaintext-ciphertext pairs. Hence, for an adaptively chosen space of $\frac{2^{n_{in}}}{2^c}$ (i.e., $\frac{2^{n_{in}} \cdot n_{in}}{2^c}$ many bits) many table entries available to $\mathcal{A}_{ACS}$, we have $2^{-c} \geq 1 - e_{n_{in}}^{AV}$, and hence $A \geq \frac{\log_{e_{n_{in}}}(1-2^{-c})}{V}$. In this case, the probability that a random plaintext is included in the set of A computed plaintext-ciphertext pairs is $A \cdot 2^{-128}$. Otherwise, if the challenge random plaintext is not in the list of A plaintext-ciphertext, given a space of size $\frac{2^{n_{in}}}{2^c}$, the probability that the random plaintext can be correctly encrypted is $p_1(1 - A \cdot 2^{-128}) \leq p_1$. Note that $\mathcal{A}_{ACS}$ has already adaptively computed a list of A plaintext-ciphertext pairs (using the adaptive leakages), and hence for any plaintext outside this set, the ciphertext will be computed with probability p_1 in a non-adaptive way and p_1 can be calculated in a similar way as in Theorem 4. $\square$

5.2.1 Space-Hardness Bounds for EWEMx Instances under ACS

- $\text{EWEMx}^{32}[2, 2, 2, 1, 8, 1, 4, \text{AES}_1, \text{AES}_1, \text{AES}_6, \text{AES}_1]$: $(\frac{2^{32}}{2^{1.067}}, 102.05)$ space-hardness,
- $\text{EWEMx}^{16}[2, 2, 2, 1, 8, 1, 8, \text{AES}_1, \text{AES}_1, \text{AES}_6, \text{AES}_1]$: $(\frac{2^{16}}{2^{0.55}}, 118.10)$ space-hardness.

Here, we recommend only two instances of EWEMx with their ACS bounds, for the sake of simplicity. Moreover, the ACS bounds for two additional instances are given in Table 1.

Table 2: Summary of Hybrid Code-lifting Attack on EWEMx for quarter size of code leakage

Cipher	Attack Technique for 2 nd step of Black-box Phase	Complexity
EWEMx ³² [2, 2, 2, 1, ℓ , 1, 4, AES ₁ , AES ₁ , AES ₆ , AES ₁]	Simple Procedure as [32]	2 ^{87.87}
EWEMx ¹⁶ [2, 2, 2, 1, ℓ , 1, 8, AES ₁ , AES ₁ , AES ₆ , AES ₁]	Filter-then-guess as [32]	2 ^{109.21}

5.3 Hybrid Code-Lifting Attack for EWEMx

In this attack, we no longer restrict the adversary to a non-adaptive, read-only model. We assume the adversary is adaptive. In the attack scenario, a white-box attacker leaks the implementation, and this leakage is used by a black-box adversary. If the target cipher is space-hard, it contains large amounts of information about the secret even after the leakage. For example, the code size for EWEMx is $2^{n_{in}} \cdot n_{in}$. Assuming the quarter size of the code is leaked, the remaining $\frac{3 \cdot 2^{n_{in}} \cdot n_{in}}{4}$ bits is also huge compared to the usual block cipher implementations using 128 or 256-bit secret key for $n_{in} = 16, 32$. Thus, intuitively, it is not easy for the black-box attacker to retrieve the full program using the leaked data. For EWEMx, we assume that the adversary can read all table entries and can access the table adaptively. Hence, the adversary can leak the entries of multi-iteration of Sboxes. If the adversary leaks the entries of the table $T_{n_{in}}$, then the adversary also gets almost the same number of entries of SuperSbox. Precisely, if the adversary gets N many $T_{n_{in}}$ input-output pairs, she can obtain a maximum $N - \ell + 1$ many SuperSbox input-output pairs, which is very close to N for a large N . Thus, to provide an upper bound on the attack complexity, we consider the SuperSbox as one Sbox, and hence consider it as one $T_{n_{in}}$ input-output pair. Thus, to calculate the attack complexity considering EWEMx with r_1, r_2 , and r_3 , we simply compute the complexity of $(r_1 + 1) + (r_2 + 1) + (r_3 + 1) - 1$ round EWEMr with $\ell = 1$. The only exception is non-uniform random permutations within the rounds (the permutations are π, π_1, π_2 or π_3). However, the choice of permutations does not impact this calculation if the overall construction is black-box secure and the underlying permutations are sufficiently strong. Hence, we compute the complexity of EWEMr ^{n_{in}} [$r_1 + r_2 + r_3 + 2, 1, w, \pi$]. The calculation is exactly the same as SPNbox with $(r_1 + r_2 + r_3 + 3)$ rounds (since EWEMr with $(r_1 + r_2 + r_3 + 2)$ rounds corresponds to SPNbox with $(r_1 + r_2 + r_3 + 3)$ rounds as both of them have $(r_1 + r_2 + r_3 + 3)$ iterations of Sboxes).

To recover a huge amount of secret information (i.e., table entries), hybrid code-lifting aims to recover a part of (unleaked) table entries by the white-box adversary. The exact attack procedure depends on the target cipher. For example, the *guess-and-determine* approach has been used for SPNbox. As mentioned before, EWEMr has a similar structure with $\ell = 1$ and Sbox components are similar, providing security confidence, and the masking blockciphers do not leak any data about the Sbox interfaces. Hence, we apply the same technique on EWEMx. We use the *Coupon Collector's* problem to analyze the cipher. In many cases, randomly picked table entries are recovered. Precisely, the attack often reveals already known table entries. This observation is closely related to the coupon collector's problem, such that the goal is to collect all table entries acting as coupons. We follow exactly the same procedure as described in [32], and hence, we skip the details due to the shortage of space. The attack complexities for two of the EWEMx ^{n_{in}} instances are given in Table 2. For the rest of the instances, we observe a similar result.

6 Specification of EWEMrl and EWEMxl

We now describe the specification of the **E**xtended **W**hite-box **E**ven **M**ansour with longevity against read-only malware (EWEMrl) construction, that achieves longevity under continuous leakage. This is one of the most important parts of this paper. To the best of our knowledge, EWEMrl is currently the only construction that achieves a significant level of longevity. EWEMrl is a block cipher with the block length n -bits with a κ -bit secret key. We first specify the black-box description of EWEMrl below.

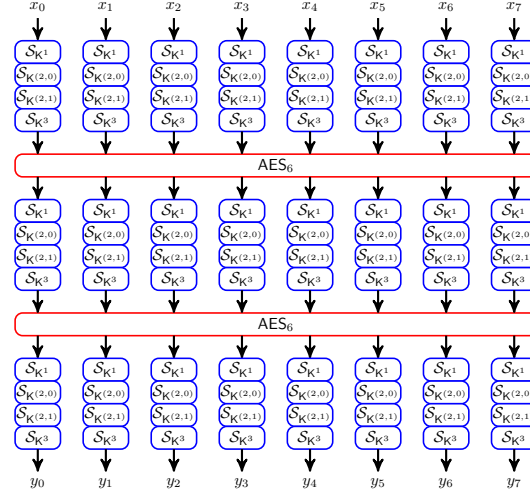


Figure 8: EWEMrl¹⁶ Black-box Encryption Algorithm for $r = 2, \ell = 4, w = 8, \pi = AES_6, b = 01$

Algorithm 4 Subroutine $\chi_K^{\ell, w, b}$. Note that, $\text{parse}_w(X)$ divides X into w , n_{in} -bit words. Also, $K = K^{(1)} \| K^{(2, b_0)} \| \dots \| K^{(2, b_{\ell-3})} \| K^{(3)}$,

Input: X
 1: $X_0 \| X_1 \| \dots \| X_{w-1} \leftarrow \text{parse}_w(X)$
 2: **return** $(S^\ell)_K(X_0) \| (S^\ell)_K(X_1) \| \dots \| (S^\ell)_K(X_{w-1})$, where $(S^\ell)_K(X) = S_{K^{(3)}} \circ S_{K^{(2, b_{\ell-3})}} \circ S_{K^{(2, b_{\ell-4})}} \circ \dots \circ S_{K^{(2, b_0)}} \circ S_{K^{(1)}}(X)$

6.1 Black-box Description of EWEMrl

EWEMrl in Algorithm 5 is structurally similar to EWEMr, except they differ in key K defined in Algorithm 2. The internal structure of EWEMr supports code updates without changing the functionality. In the case of EWEMr, $K = K_0 \| K_0 \| \dots \| K_0$. However, EWEMrl simply changes K from $K_0 \| K_0 \| \dots \| K_0$ to $K^{(1)} \| K^{(2, b_0)} \| K^{(2, b_1)} \| \dots \| K^{(2, b_{\ell-3})} \| K^{(3)}$, where $b_i \in \{0, 1\}$, for all $0 \leq i \leq \ell-3$. Thus, we need to add a parameter $b \in \{0, 1\}^{\ell-2}$ in the description of EWEMrl. EWEMrl black-box description is given in Fig.8.

The Choice of the Sbox and π : For the concrete proposals, we use the similar small SPN based block cipher for Sbox from [8] as we proposed for EWEMr. The description is given in Section 3.1.1. Furthermore, for all of our proposed instances, we use AES_6 as π .

The parameter b : We use b to denote the choice of the Sboxes in the *iteration*-Sbox chain. As an example, from Fig.8, in $S_{K^{(3)}} \circ S_{K^{(2,0)}} \circ S_{K^{(2,1)}} \circ S_{K^{(1)}}$, $b = 01$, as b corresponds to $S_{K^{(2,0)}} \circ S_{K^{(2,1)}}$ between $S_{K^{(3)}}$ and $S_{K^{(1)}}$. Similarly, when $b = 0011$, it corresponds to $S_{K^{(3)}} \circ S_{K^{(2,0)}} \circ S_{K^{(2,0)}} \circ S_{K^{(2,1)}} \circ S_{K^{(2,1)}} \circ S_{K^{(1)}}$.

6.2 White-box Description of EWEMrl

In the white-box implementation of EWEMrl, we replace the non-linear layer S_K with table look-up. This is done to protect against *key-extraction* and *code-lifting attacks*. Additionally, we aim to achieve *longevity*. One approach to achieve longevity is by updating the tables used for non-linear layers. To update any table *securely*, without changing the embedded key in the small key-dependent Sbox, we can mask the input and/or output of each table. However, it is important to choose the masking in a way that it does not alter the original functionality of the encryption algorithm. If

Algorithm 5 $\text{EWEMr}_{\mathbf{K}}^{n_{in}}[r, \ell, w, \pi, b]$ Black-box Description

-
- 1: Input: X
 - 2: $K^{(1)} \| K^{(2, b_0)} \| K^{(2, b_1)} \| \dots \| K^{(2, b_{\ell-3})} \| K^{(3)} \leftarrow \text{parse}_{\ell}(K)$
 - 3: $X_0 \| X_1 \dots \| X_{w-1} \leftarrow \text{parse}_w(X)$
 - 4: **return** $\underbrace{\chi_K^{\ell, w, b} \circ \pi \circ \chi_K^{\ell, w, b} \circ \dots \circ \pi \circ \chi_K^{\ell, w, b}}_{r\text{-times}}(X)$
-

two tables T_1 and T_2 are accessed sequentially, the mask added at the output of T_1 should be canceled at the input of T_2 . This concept was first proposed in [26] for the Yoro cipher. In the following, we explain how to obtain updatable tables corresponding to the non-linear operations of EWEMr.

At first, it is important to note that $\chi_K^{\ell, w, b}$ either use the three Sboxes $\mathcal{S}_{K^{(1)}}$, $\mathcal{S}_{K^{(2,0)}}$ and $\mathcal{S}_{K^{(3)}}$ (if b is a zero string) or all four Sboxes $\mathcal{S}_{K^{(1)}}$, $\mathcal{S}_{K^{(2,0)}}$, $\mathcal{S}_{K^{(2,1)}}$ and $\mathcal{S}_{K^{(3)}}$. Thus, we need **at most** four tables, denoted as $T_{n_{in}}^{(1)}$, $T_{n_{in}}^{(2,0)}$, $T_{n_{in}}^{(2,1)}$, and $T_{n_{in}}^{(3)}$ to replace $\mathcal{S}_{K^{(1)}}$, $\mathcal{S}_{K^{(2,0)}}$, $\mathcal{S}_{K^{(2,1)}}$ and $\mathcal{S}_{K^{(3)}}$, respectively. Furthermore, to ensure longevity, we mask the input and/or output of each table using one encryption algorithm E and its decryption D such that (1) $T_{n_{in}}^{(1)}$ corresponds to $E_{K^*} \circ \mathcal{S}_{K^{(1)}}$, (2) $T_{n_{in}}^{(2,0)}$ corresponds to $E_{K^*} \circ \mathcal{S}_{K^{(2,0)}} \circ D_{K^*}$, (3) $T_{n_{in}}^{(2,1)}$ corresponds to $E_{K^*} \circ \mathcal{S}_{K^{(2,1)}} \circ D_{K^*}$, and (4) $T_{n_{in}}^{(3)}$ corresponds to $\mathcal{S}_{K^{(3)}} \circ D_{K^*}$, where K^* is sampled independently from the master-key K . Note that, to achieve longevity, the underlying E_{K^*} , and D_{K^*} are updated. Let us denote the lookup tables after the u^{th} update as

$$(T_{n_{in}}^{(1)})^u, (T_{n_{in}}^{(2,0)})^u, (T_{n_{in}}^{(2,1)})^u, \dots, (T_{n_{in}}^{(2, b_{\ell-3})})^u, (T_{n_{in}}^{(3)})^u.$$

Note that EWEMr will remain functionally the same after any update due to

$$(T_{n_{in}}^{(3)})^u \circ (T_{n_{in}}^{(2, b_{\ell-3})})^u \circ \dots \circ (T_{n_{in}}^{(2, b_0)})^u \circ (T_{n_{in}}^{(1)})^u = T_{n_{in}}^{(3)} \circ T_{n_{in}}^{(2, b_{\ell-3})} \circ \dots \circ T_{n_{in}}^{(2, b_0)} \circ T_{n_{in}}^{(1)},$$

for any u . Thus, update tables also computes $\mathcal{S}_{K^{(3)}} \circ \mathcal{S}_{K^{(2, b_{\ell-3})}} \circ \mathcal{S}_{K^{(2, b_{\ell-4})}} \circ \dots \circ \mathcal{S}_{K^{(2, b_0)}} \circ \mathcal{S}_{K^{(1)}}$ even if E_{K^*} , and D_{K^*} are updated. We denote the white-box implementation of $\chi_K^{\ell, w, b}$ (see Algorithm 4) as $\Sigma^{\ell, w, b}$.

6.2.1 Rationale behind b

In EWEMr, we use an $\ell - 2$ -bit string b to determine the choice for look-up tables. We show that when $b = 0$, EWEMr with larger Sboxes can achieve longevity. We also show that if the choice of $b \neq 0$ follows certain conditions, EWEMr can achieve a sufficient longevity bound for both the 16-bit and 32-bit versions. Theoretically, we can view each bit of b as a 1-bit tweak, and we can achieve longevity by adjusting the tweak bits. To be precise, due to the encryption-decryption canceling structure of EWEMr described in Section 6.2, we need to use multiple tables, and hence we have the liberty to use independent core tables $\mathcal{S}_{K^{(1)}}$, $\mathcal{S}_{K^{(2,0)}}$, $\mathcal{S}_{K^{(2,1)}}$, and $\mathcal{S}_{K^{(3)}}$. Hence, we would like to use as many tables as possible to achieve independence of the tables. Also, we can use different combinations of chains to avert the issues with fixed points and cycles (the combinations of b 's are detailed in Section 6.6). The choice of b described by Condition 1 ensures that, even if there are cycles in one table, they are thwarted by the invocation of another independent table depending on the corresponding bit value for b . Hence, the choice of b is very important in this design. Note that we need to ensure that there can not exist cycles with length less than or equal to ℓ , because if there are cycles with length more than ℓ , they won't be detected (for cycles see Property 1). The choice of b following Condition 1 described later (see Section 6.6) ensures that.

6.3 Space-hardness Bounds of EWEMr

We follow the **same attack model** as in Section 1.5. EWEMr is structurally the same as EWEMr, except that it uses three/four different tables, while EWEMr uses

only one table. It is easy to see that EWEMrl achieves at least the same level of space-hardness as EWEMr. In fact, in this case, the number of possible collisions is less than that in EWEMr, which in turn may provide a better space-hardness bound than EWEMr. Note that, in this case, we are actually depicting that the value of Z in Definition 4 for EWEMrl is lower bounded by that for EWEMr. This means for EWEMrl, the probability of encrypting a random plaintext is upper bounded by the same for EWEMr. Hence, we are not providing any proof for the space-hardness bounds for EWEMrl. The detailed bounds are given in Table 1.

6.4 Hybrid Code-Lifting Attack for EWEMrl

EWEMrl uses two different tables $T_{n_{in}}^{(1)}$ in the first position and $T_{n_{in}}^{(3)}$ in the last position of the first and last rounds, respectively, for any b . Similar to the technique for EWEMr, the attacker can retrieve some entries of the table $T_{n_{in}}^{(3)}$ with the encryption query in the first step of black-box phase, instead of the entries of $T_{n_{in}}^{(1)}$. Thus, it is hard to fulfill the attacker's goal of recovering all table entries. Hence EWEMrl is secure against this attack for any b .

6.5 Upper Bound on the Longevity of EWEMrl with $b = 00 \dots 0$

We start with the following observation.

Observation 3. *For the sake of simplicity, let us denote the sequence of tables $(T_{n_{in}}^{(1)})^u, (T_{n_{in}}^{(2,b_0)})^u, (T_{n_{in}}^{(2,b_1)})^u, \dots, (T_{n_{in}}^{(2,b_{\ell-3})})^u, (T_{n_{in}}^{(3)})^u$ by $F_0^u, F_1^u, \dots, F_{\ell-2}^u, F_{\ell-1}^u$ respectively. We observe that $F_j^u \circ F_{j-1}^u \circ \dots \circ F_i^u$, for any $i < j \in \{0, 1, \dots, \ell-1\}$, and $(i, j) \neq (0, \ell-1)$ (for $(i, j) = (0, \ell-1)$, the composition will be invariant for any update), is independent of $F_j^{u'} \circ F_{j-1}^{u'} \circ \dots \circ F_i^{u'}$, when $u \neq u'$.*

This implies the function $F_{\ell-1}^u \circ \dots \circ F_0^u$ will be functionally the same across updates, but any strict sub-sequence $F_j^u \circ F_{j-1}^u \circ \dots \circ F_i^u$ has either of the following structures: $E_{K^*} \circ S$ or $S \circ D_{K^*}$ or $E_{K^*} \circ S \circ D_{K^*}$. Hence, when the code (table) updates E_{K^*} , and D_{K^*} , the new code (table) becomes independent due to the outer layer E_{K^*} , and the inner layer D_{K^*} .

We first propose Property 1 and then Theorem 6 below.

Property 1. We say the table T has the property **noCycle**, if the permutation from $\{0, 1\}^{n_{in}}$ to $\{0, 1\}^{n_{in}}$ represented by T does not have any i -length cycle with $1 \leq i \leq \ell-2$. If T satisfies **noCycle**, we say $\text{noCycle}(T) = 1$, otherwise $\text{noCycle}(T) = 0$. **For example**, we say T has a 3-cycle if $\exists x \in \{0, 1\}^{n_{in}}$ such that, $T^3(x) = x$.

Theorem 6. *Let Property 1 holds for $T_{n_{in}}^{(2,0)}$ across all the updates. For any $1 \leq u \leq 2^{c\ell-n_{in}}$, there exists no adversary $\mathcal{A}$ that, given at most $\frac{2^{n_{in}}}{2^c}$ (each of the tables has $2^{n_{in}}$ input-output pairs) many input-output pairs **non-adaptively** from each of the three tables $T_{n_{in}}^{(1)}, T_{n_{in}}^{(2,0)}, T_{n_{in}}^{(3)}$ during every update until the u^{th} update, can determine $(T^{(0)})^u(P_0)$ for a random P_0 with probability more than $2^{-2c}q^{\ell-2}$, where $q = 2^{-c}$. This holds under the assumption that $\forall j, 1 \leq j \leq u-1$, $\mathcal{A}$ is given insufficient number of entries from each of $(T_{n_{in}}^{(1)})^j, (T_{n_{in}}^{(2,0)})^j$, and $(T_{n_{in}}^{(3)})^j$ to successfully determine $(T^{(0)})^j(P_0)$ (note that, $(T^{(0)})^u(P_0) = (T^{(0)})^j(P_0)$) for a random P_0 with a probability more than $2^{-2c}q^{\ell-2}$ (this assumption is actually implied from the amount of leakage given to the adversary. We just mentioned it for an easy understanding).*

Proof: We first justify why we are considering determining only $(T^{(0)})^u(P_0)$ (we can view P_0 as the first n_{in} -bit chunk of a random plaintext $P = P_0 \| P_1 \| \dots \| P_{w-1}$) instead of determining the ciphertext C from P .

- Note that, we at least need to find the probability of determining $(T^{(0)})^u(P_0)$ (similar to evaluating the first column of Fig.3) under code (table) updates. This probability eventually provides an upper bound on longevity, as longevity corresponds to the probability of computing C for P under continuous leakage,

such that to compute C , we need to multiply the probabilities for determining the other columns given the value of previous columns (i.e., $(T^{(0)})^u(P_1)$, $(T^{(0)})^u(P_2)$, and so on) with the probability of determining $(T^{(0)})^u(P_0)$.

- In fact, determining $(T^{(0)})^u(P_0)$ is the minimum amount of computations the adversary must make to leak one functionally constant information from the implementation. This comes from Observation 3, which says that, any partial function of $(T^{(0)})^u(P_0)$ are not functionally constant but $(T^{(0)})^u(P_0)$ is functionally constant (i.e., $(T^{(0)})^u(P_0) = (T^{(0)})^{u'}(P_0)$, for any $u \neq u'$). Hence, it is sufficient to upper bound the probability of computing $(T^{(0)})^u(P_0)$ for a random P_0 .

This eventually makes our proof simple. We present the following points regarding the calculation of longevity bound.

- The main idea of longevity stems from the fact that leakages from a table during previous updates can not compress the table in the latest update. In our construction, when $u' < u$, then $(T)^u$ and $(T)^{u'}$ such that $(T)^u \in \{(T_{n_{in}}^{(1)})^u, (T_{n_{in}}^{(2,0)})^u, (T_{n_{in}}^{(3)})^u\}$, and $(T)^{u'} \in \{(T_{n_{in}}^{(1)})^{u'}, (T_{n_{in}}^{(2,0)})^{u'}, (T_{n_{in}}^{(3)})^{u'}\}$ are independent. Hence leakage from $(T)^u$ does not depend on the leakage from $(T)^{u'}$. Given the assumption that $\mathcal{A}$ has access to insufficient table entries to compute $(T^{(0)})^j(P_0)$ for a random P_0 with a probability more than $2^{-2c}q^{\ell-2}$, and these entries do not help to compress the table during u^{th} update, $\mathcal{A}$ has to start from scratch to compute $(T^{(0)})^u(P_0)$ using just the entries from the tables during the u^{th} update. Below, we mention the two possible ways to compute $(T^{(0)})^u(P_0)$.
- **First way (Guessing won't work):** $(T^{(0)})^u(P_0)$ can be guessed with the probability $2^{-n_{in}}$, but to determine the correct guess, we need to compute all the other column operations (each column by guessing or ℓ table look-ups) corresponding to the ciphertext computation for random P , making the probability of success negligible (actually at most 2^{-n}). In fact, it is better to guess the whole ciphertext with the probability 2^{-n} . Hence, we will omit this case.
- **Second way:** $(T^{(0)})^u(P_0)$ can be computed through table look-ups. Determining $(T^{(0)})^u(P_0)$ relies on a sequence of matching table lookups, with success probability bounded by the product of matching probabilities across ℓ iterations. As Property 1 holds for $(T_{n_{in}}^{(2,0)})^u$, lookup inputs remain distinct and there is no collision for the middle $\ell - 2$ compositions of $(T_{n_{in}}^{(2,0)})^u$, leading to a probability bound of $2^{-2c}q^{\ell-2}$ (similar to Case 1 but without column collision).

We now elaborate on the second way. From Property 1, there are no $1 \leq i \leq \ell - 2$ length cycles in $(T_{n_{in}}^{(2,0)})^u$. Hence, $(T^{(0)})^u(P_0)$ can be determined by traversing through all the ℓ table look-ups. Note that, all the $\ell - 2$ table lookup inputs to $(T_{n_{in}}^{(2,0)})^u$ are distinct (from Property 1). Hence, following the same idea discussed in Theorem 1 regarding the probability q , the $\ell - 2$ table lookups can be successfully done with the probability $q^{\ell-2}$. The lookups to $(T_{n_{in}}^{(1)})^u$, and $(T_{n_{in}}^{(3)})^u$ can be successfully done with the probability 2^{-2c} . Hence, the probability of a successful traversal through $T^{(b)}$ is bounded by $2^{-2c}q^{\ell-2}$.

Bound on u : Data leakage occurs by determining a valid (X, Y) pair with $Y = (T^{(0)})^u(X)$ because this pair is functionally constant across the updates. Since Y can be computed with probability $2^{-2c}q^{\ell-2} = 2^{-c\ell}$ for a random X , and there are $2^{n_{in}}$ possible inputs, the probability of leaking one such functionally constant input-output pair from $(T^{(0)})^u$ is $2^{n_{in}-c\ell}$. Equivalently, leakage of one input-output pair is expected after every $2^{c\ell-n_{in}}$ updates, making u upper bounded by $2^{c\ell-n_{in}}$. $\square$

EWEMrl with $b = 00 \dots 0$ remains secure if Property 1 holds for $(T_{n_{in}}^{(2,0)})^u$. However, it is the server's responsibility to enforce this. This update process has high complexity. So we later introduce EWEMrl with $b \neq 00 \dots 0$, which relies on a relaxed

assumption (Assumption 1) and significantly reduces server overhead (discussed in Section 6.7.3).

6.6 Upper Bound on the Longevity of EWEMrl with $b \neq 00 \dots 0$

We now compute the upper bound for longevity below. We propose an assumption and a condition below. Given that the code satisfies this assumption and condition, we will show the bound on the longevity. For $\mathbf{b} = \mathbf{b}_0 \mathbf{b}_1 \dots \mathbf{b}_{\ell-3}$,

$$\mathbf{T}_2^{(\mathbf{b})} = (\mathbf{T}_{n_{in}}^{(2, \mathbf{b}_{\ell-3})})^u \circ (\mathbf{T}_{n_{in}}^{(2, \mathbf{b}_{\ell-4})})^u \dots \circ (\mathbf{T}_{n_{in}}^{(2, \mathbf{b}_0)})^u$$

which is the composition of the inner layer containing $(\mathbf{T}_{n_{in}}^{(2,0)})^u$ and $(\mathbf{T}_{n_{in}}^{(2,1)})^u$. Let $q = 2^{-c}$ be the probability bound of a successful table lookup. Let us first define some terms to understand our design. We define the *coverage* of a probability p as follows.

Definition 9. Coverage for probability p is the maximum integer t , such that $q^t \geq p$ (the transition probability for traversing t tables with no collision is greater than or equal to p). In this case, we say that p covers t many table lookups.

From the definition, we can say that $2^{-n_{in}}$ covers $t = \lfloor \frac{n_{in}}{c} \rfloor$ many table lookups. We next define what a collision length is.

Definition 10. Let there is a collision in the inputs of $(\mathbf{T}_{n_{in}}^{(2, b_i)})^u$, and $(\mathbf{T}_{n_{in}}^{(2, b_{i+v})})^u$, then v is denoted as the length of the collision. We say that the **collision occurred** at $b_i b_{i+1} \dots b_{i+v}$, or equivalently the string $b_i b_{i+1} \dots b_{i+v-1}$ corresponds to the collision. We also say that b_{i+v} is the **receiving point of the collision** (this signifies that the input to $(\mathbf{T}_{n_{in}}^{(2, b_{i+v})})^u$ collides with the input to some previous table).

We propose the following assumption needed for this construction.

Assumption 1. $(\mathbf{T}_{n_{in}}^{(2,0)})^u$, and $(\mathbf{T}_{n_{in}}^{(2,1)})^u$ can not have the same fixed point together (i.e., $\nexists X$ such that $(\mathbf{T}_{n_{in}}^{(2,0)})^u(X) = (\mathbf{T}_{n_{in}}^{(2,1)})^u(X) = X$).

For $\mathbf{b} \neq 000 \dots 0$, four tables are used in the u^{th} code (table) update phase: $(\mathbf{T}_{n_{in}}^{(1)})^u$, $(\mathbf{T}_{n_{in}}^{(2,0)})^u$, $(\mathbf{T}_{n_{in}}^{(2,1)})^u$, and $(\mathbf{T}_{n_{in}}^{(3)})^u$. The idea is that if there is a collision in the input to any of the $(\mathbf{T}_{n_{in}}^{(2,0)})^u$ inputs, the collision chain can be broken by invoking $(\mathbf{T}_{n_{in}}^{(2,1)})^u$ (similar as domain separation) and the table lookup gets randomized through this transition. This keeps server overhead low, requiring only the assumption that no $x \in \{0, 1\}^{n_{in}}$ is a fixed point for both $(\mathbf{T}_{n_{in}}^{(2,0)})^u$ and $(\mathbf{T}_{n_{in}}^{(2,1)})^u$, which happens with negligible probability $2^{-n_{in}}$. Together, this assumption and the choice of $\mathbf{b}$ ensure an extremely low chance of traversing $\mathbf{T}^{(\mathbf{b})}$.

EWEMrl with $\mathbf{b} \neq 00 \dots 0$ is secure if Assumption 1 holds true. To ensure this assumption holds, the server overhead is very small (only one additional table update after every $2^{n_{in}}$ table updates) as discussed in Section 6.7.3. We now mention the condition on $\mathbf{b}$ as follows.

Condition 1. Let $t = \lfloor \frac{n_{in}}{c} \rfloor$. For any sub-string β of $\mathbf{b}$, if $\beta \parallel \beta'$ is also a sub-string of $\mathbf{b}$, with $|\beta'| = t + 1$, then the $t + 1$ -length prefix of $\underbrace{\beta \parallel \beta \parallel \dots \parallel \beta}_{i \text{ times}}$, such that i is the minimum integer with $|\underbrace{\beta \parallel \beta \parallel \dots \parallel \beta}_{i \text{ times}}| \geq t + 1$ can not be equal to β' .

Rationale behind Condition 1. Condition 1 captures the scenario when β corresponds to a collision. There can be two cases.

- **Case 1.** $|\beta| \leq t$. Let $\beta = \mathbf{b}_i \mathbf{b}_{i+1} \dots \mathbf{b}_{i+v-1}$. In that case, $\mathbf{b}_{i+v}$ is the receiving point of the collision. Hence, the collision probability $2^{-n_{in}}$ covers the next t table look-ups starting from $(\mathbf{T}_{n_{in}}^{(2, b_{i+v})})^u$ to $(\mathbf{T}_{n_{in}}^{(2, b_{i+v+t-1})})^u$. On the other hand

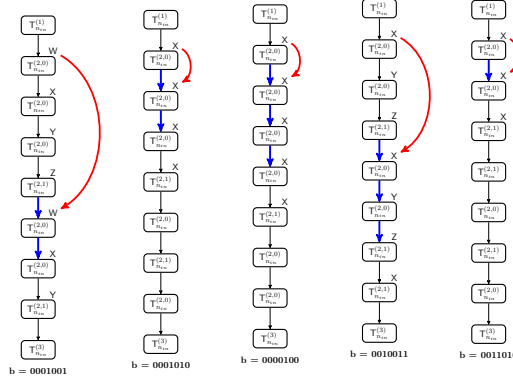


Figure 9: Choices of $\mathbf{b}$ and their validity. Here $\ell = 9$, $c = 6$, $n_{in} = 16$. Red arrows denote collision with probability $2^{-n_{in}}$, and blue arrows denote table look-ups without transition probability

if the $t+1$ -length prefix of $\underbrace{\beta\|\beta\|\dots\|\beta}_{i \text{ times}}$ is equal to β' , then using the table lookups corresponding to β , we can make all the $t+1$ table look-ups corresponding to β' . Hence, there are $t+1$ table lookups with probability 1, and they correspond to the input to $(T_{n_{in}}^{(2,b_i+v)})^u$, $(T_{n_{in}}^{(2,b_i+v+1)})^u, \dots, (T_{n_{in}}^{(2,b_i+v+t)})^u$, respectively. Hence, $2^{-n_{in}}$ cannot cover this $t+1$ table look-ups as $2^{-n_{in}} > q^{t+1}$, and thus the total determining probability for $T_2^{(b)}$ will be more than $q^{\ell-2}$.

- **Case 2.** $|\beta| > t$. The argument is exactly similar in this case. In fact, this is a sub-case of Case 1, with $i = 1$.

Examples of $\mathbf{b}$: Using Fig.9, we describe the validity of different values of $\mathbf{b}$'s with $\ell = 9$, $c = 6$, and $n_{in} = 16$ for the sake of explanation. For all the above cases in Section 6.6, $t = \lfloor \frac{16}{6} \rfloor = 2$.

1. **$\mathbf{b} = 0001001$.** In this example, there is a collision in the inputs to $(T_{n_{in}}^{(2,b_0)})^u$, and $(T_{n_{in}}^{(2,b_4)})^u$. Now $\beta = b_0b_1b_2b_3 = 0001$, and $\beta' = b_4b_5b_6 = 001$. In this case, β' is not a prefix of β . Hence, β is valid. However, if $\beta = b_1b_2b_3 = 001$, then $\beta' = b_4b_5b_6 = 001$. Hence, the second choice of β fails the test and if $b_1b_2b_3$ corresponds to a collision, then $b_4b_5b_6$ also corresponds to a collision, and the internal table lookup values are the same as β and happen with probability 1.
2. **$\mathbf{b} = 0001010$.** In this case, all possible β are valid. Take $\beta = b_0 = 0$, and $\beta' = b_1b_2b_3 = 001$. Here β' is not a prefix of $\beta\|\beta\|\beta$. Similarly, any β in $\mathbf{b}$ is valid.
3. **$\mathbf{b} = 0000100$.** Let $\beta = b_0 = 0$, then $\beta' = b_1b_2b_3 = 000$. In this example, β' is a prefix of $\beta\|\beta\|\beta$. Hence, this $\mathbf{b}$ is not valid.
4. **$\mathbf{b} = 0010011$.** If $\beta = b_0b_1b_2 = 001$ and $\beta' = b_3b_4b_5 = 001$. In this example, β' is a prefix of β . Hence, this β is not valid.
5. **$\mathbf{b} = 0011010$.** In this case, all possible β is valid. Take $\beta = b_0 = 0$, and $\beta' = b_1b_2b_3 = 011$. Here β' is not a prefix of $\beta\|\beta\|\beta$. Similarly, any β in $\mathbf{b}$ is valid.

We will now present Theorem 7 below to upper bound the longevity.

Theorem 7. *Let Assumption 1 be true across all the updates, and $\mathbf{b}$ follows Condition 1. For any $1 \leq u \leq 2^{c\ell-n_{in}}$, there exists no adversary $\mathcal{A}$ that, given at most $\frac{2^{n_{in}}}{2^c}$ (each of the tables has $2^{n_{in}}$ input-output pairs) many input-output pairs **non-adaptively** from each of $T_{n_{in}}^{(1)}, T_{n_{in}}^{(2,0)}, T_{n_{in}}^{(2,1)}, T_{n_{in}}^{(3)}$ during every update until the u^{th} update, can determine $(T^{(0)})^u(P_0)$ for a random P_0 with probability more than $2^{-2^c}q^{\ell-2}$, where $q = 2^{-c}$. This holds under the assumption that $\forall j, 1 \leq j \leq u-1$, $\mathcal{A}$ is given*

insufficient number of entries from each of $(\mathsf{T}_{n_{in}}^{(1)})^j$, $(\mathsf{T}_{n_{in}}^{(2,0)})^j$, $(\mathsf{T}_{n_{in}}^{(2,1)})^j$, and $(\mathsf{T}_{n_{in}}^{(3)})^j$ to determine $(\mathsf{T}^{(0)})^j(P_0)$ (note that, $(\mathsf{T}^{(0)})^u(P_0) = (\mathsf{T}^{(0)})^j(P_0)$) for a random P_0 with a probability more than $2^{-2c}q^{\ell-2}$.

Proof: We follow the same **simple** strategy of upper bounding $(\mathsf{T}^{(b)})^u(P_0)$ for a random P_0 . Similar to Theorem 6, previous leakages do not help to compress tables during the latest update. Similarly, given the assumption that $\mathcal{A}$ has access to insufficient table entries to compute $(\mathsf{T}^{(0)})^j(P_0)$ for a random P_0 with a probability more than $2^{-2c}q^{\ell-2}$, $\mathcal{A}$ has to start from scratch to compute $(\mathsf{T}^{(0)})^u(P_0)$. Thus, we just need to show that the probability of determining $\mathsf{C}'_0 = (\mathsf{T}_{n_{in}}^{(2,b_{\ell-3})})^u \circ (\mathsf{T}_{n_{in}}^{(2,b_{\ell-4})})^u \circ \dots \circ (\mathsf{T}_{n_{in}}^{(2,b_0)})^u(P'_0)$, for a random P'_0 is bounded by $q^{\ell-2}$. If this is true, then by the table lookup probability 2^{-c} for $(\mathsf{T}_{n_{in}}^{(1)})^u$, and 2^{-c} for $(\mathsf{T}_{n_{in}}^{(3)})^u$, we get the desired bound, which is $2^{-2c}q^{\ell-2}$.

We take the approach of traversing $\mathbf{b}$ from $\mathbf{b}_0$ to $\mathbf{b}_{\ell-3}$ and check whether the corresponding $\mathbf{b}_i$ is the receiving point of a collision or not. If $\mathbf{b}_i$ is the receiving point of a collision, then we skip to $\mathbf{b}_{i+t}$ as the intermediate table look-ups are covered by the collision probability. Else, move to $\mathbf{b}_{i+1}$. We show the steps and count the corresponding probability below. Let, $t = \lfloor \frac{n_{in}}{c} \rfloor$

1. Check $\mathbf{b}_0$. Note that $\mathbf{b}_0$ can not be a receiving point of a collision as it corresponds to the first table look-up for $(\mathsf{T}_{n_{in}}^{(2,b_0)})^u$. Hence, $\mathbf{b}_0$ corresponds to the transition probability q for the table $(\mathsf{T}_{n_{in}}^{(2,b_0)})^u$
2. Check $\mathbf{b}_i$ ($1 \leq i \leq \ell - 3$). We have two cases. If $\mathbf{b}_i$ is **not a receiving point** of a collision, then it contributes q to the total probability and we will move to $\mathbf{b}_{i+1}$. If $\mathbf{b}_i$ is **the receiving point** of a collision, then **Condition 1** ensures that from $\mathbf{b}_i$, the maximum number of table look-ups with probability 1 is t . The collision probability $2^{-n_{in}}$ covers the table look-ups for $\mathbf{b}_i, \dots, \mathbf{b}_{i+t-1}$, and we will move to $\mathbf{b}_{i+t}$. Thus, the loss of probability for this t table look-ups are compensated by the collision. Hence, the probability for this t table look-ups is upper bounded by $q^t \geq 2^{-n_{in}}$.
3. Depending on whether $\mathbf{b}_i$ is the receiving point of a collision or not, we jump to $\mathbf{b}_{i+1}$ (when $\mathbf{b}_i$ is not the receiving point of any collision), or $\mathbf{b}_{i+t}$ (when $\mathbf{b}_i$ is the receiving point of a collision). The treatment of $\mathbf{b}_{i+1}$ or $\mathbf{b}_{i+t}$ is exactly the same as $\mathbf{b}_i$.
4. Continue with this traversal until the end.

This shows that the overall probability of determining C'_0 for a random P'_0 is bounded by $q^{\ell-2}$. Precisely, the above idea depicts that for any $i < j$, the traversal probability from $(\mathsf{T}_{n_{in}}^{(2,b_i)})^u$ to $(\mathsf{T}_{n_{in}}^{(2,b_j)})^u$ is always less than or equal to q^{j-i+1} . This is because if there is no collision at any $\mathbf{b}_i$, it just makes a normal table look-up, but if there is a collision at $\mathbf{b}_i$, the next t look-ups are skipped, but they are covered by the collision probability $2^{-n_{in}}$. Justification for the upper bound on u is the same as discussed in Theorem 6. $\square$

6.6.1 Longevity Bounds for Recommended EWEMrl Instances

Here, we recommend only three instances of EWEMrl with their longevity security bounds in Table 3, we have calculated the longevity bound for other instances, given in Table 1. Similarly, $n_{in} \in \{16, 32\}$ -bit tables and $\sigma(\mathsf{T})$ be the total code (table) size. Note that versions with $\ell = 16$ provide better longevity but lower throughput than the version with $\ell = 12$. See Table 4.

Table 3: Longevity security bounds of the recommended instances of EWEMrl

Cipher	Longevity Security
EWEMrl ³² [2, 16, 4, AES ₆ , b] b = 0000000000000000	$\left(\frac{\sigma(T)}{2^6}, 96\right)$
EWEMrl ¹⁶ [2, 12, 8, AES ₆ , b] b = 1010001110	$\left(\frac{\sigma(T)}{2^6}, 72\right)$
EWEMrl ³² [2, 16, 4, AES ₆ , b] b = 00000011111100	$\left(\frac{\sigma(T)}{2^6}, 96\right)$

6.7 How the Codes (Tables) are Updated by the Server?

We now describe how the codes(tables) are updated by the server. Note that the communication protocol aims to secure communication between a device (we also call it a client) and a server by using an encryption algorithm. The device runs the white-box implementation of the encryption function, and the server is assumed to be a trusted party running the black-box implementation of the corresponding decryption function. Initially, the server randomly samples encryption keys, generates the white-box code (by constructing the tables) for the encryption, and then distributes it to the device using a secure TLS channel. In the scenario of longevity as discussed earlier, the white-box code needs to be updated periodically without changing the functionality. The server updates the white-box code and sends it to the client using the same secure channel. In this case, the server first samples a new key uniformly (it simply needs to sample K^* as described above), and then updates the Sbox tables based on the newly sampled key K^* . Note that the server in this case is assumed to be trusted, and the attacker does not have any access to the server. Below, we provide more precise discussions for both $b = 00 \dots 0$ and $b \neq 00 \dots 0$.

6.7.1 Code (Table) Update for $b = 00 \dots 0$

From Property 1, there is no i length cycle where $1 \leq i \leq \ell - 2$ in $(T_{n_{in}}^{(2,0)})^u$ (table after the u^{th} update). This can be affirmed by the server's code update protocol. We would like to mention that this protocol works when $2^{n_{in}}$ is very large (for example, $n_{in} = 32$). We now describe the code update process below.

1. In the u^{th} code update phase, the server samples s independent randomly sampled keys $K_1^u, K_2^u, \dots, K_s^u$ from the key space of E . The server then prepares the tables $(T_1)^u = E_{K_1^u} \circ S_{K(2,0)} \circ D_{K_1^u}$, $(T_2)^u = E_{K_2^u} \circ S_{K(2,0)} \circ D_{K_2^u}$, $\dots$, $(T_s)^u = E_{K_s^u} \circ S_{K(2,0)} \circ D_{K_s^u}$.
2. During the table preparation, if all the s events $\neg \text{noCycle}((T_1)^u)$, $\neg \text{noCycle}((T_2)^u)$, $\dots$, $\neg \text{noCycle}((T_s)^u)$ occur, the server rejects all the tables. Otherwise, the server marks all the tables that satisfy the noCycle event, and uses one of the valid tables as $(T_{n_{in}}^{(2,0)})^u$, and keeps the others to use in the next updates.

We show the event $(\neg \text{noCycle}((T_1)^u) \cap \neg \text{noCycle}((T_2)^u) \cap \dots \cap \neg \text{noCycle}((T_s)^u))$ after the u^{th} update occurs with a low probability when s is high, and hence, with a high probability $\exists i$ such that $\text{noCycle}((T_i)^u)$, i.e., has no collision of length less than or equal to $\ell - 2$. Let us denote this event by noCycle_u , such that during the u^{th} code (table) update, $\exists i$, with $\text{noCycle}((T_i)^u)$ occurs.

Estimate noCycle_u . Since $2^{n_{in}}$ is large, we use the result that for a random permutation on a very large domain, the probability that the shortest cycle in a randomly chosen permutation of a large domain set has a length greater than $\ell - 2$ is $e^{-(1+\frac{1}{2}+\frac{1}{3}+\dots+\frac{1}{\ell-2})} \approx e^{-(\gamma+\ln(\ell-2))} = \frac{1}{e^{\gamma(\ell-2)}}$, where $\gamma = 0.57721$ is Euler's constant.³ Hence, in the u^{th} code (table) update, $Pr(\text{noCycle}_u) = 1 - \prod_{i=1}^s Pr(\neg \text{noCycle}((T_i)^u)) = 1 - (1 - \frac{1}{e^{\gamma(\ell-2)}})^s$. Thus, we have the following points.

³https://math.ucr.edu/home/baez/permutations/permutations_2.html

- For large s , $Pr(\text{noCycle}_u)$ is high. For example, for $\ell = 16$, and $s = 256$, $Pr(\text{noCycle}_u) = 0.9999718593$, and for $\ell = 16$, and $s = 512$, $Pr(\text{noCycle}_u) = 0.999999992$, which is high and close to 1.
- Clearly, the server needs to ensure all the noCycle_0 , noCycle_1 , noCycle_2 , and so on occur with high probability. Hence, for a large u , all the noCycle_0 , noCycle_1 , $\dots$ noCycle_u occur with the probability 0.52, when $u = 23000$, $\ell = 16$, and $s = 256$, and the probability 0.53, when $u = 8 \times 10^8$, $\ell = 16$, and $s = 512$. Note that, when u is small, all the noCycle_0 , noCycle_1 , $\dots$ noCycle_u events occur with a much higher probability (for example, the probability is 0.99921, when $u = 10^8$, $\ell = 16$, and $s = 512$, or 0.754718, when $u = 10000$, $\ell = 16$, and $s = 256$).

We want to mention that this process may reduce the key-space size by $\log r$ bits for the E. However, the master key space is not affected. We would also like to mention that the server overheads contain (1) detecting the existence of cycles with the length $\leq \ell - 2$, and (2) computing s many tables. For (1), the server can detect cycles during the table construction, and for (2), the server needs to repeat the process s times. Hence, the server needs a total of $s \cdot 2^{n_{in}}$ computations.

6.7.2 Drawbacks of the above Table Update Technique

The above code (table) update has three drawbacks (1) This process works well mainly for large n_{in} , (2) noCycle does not occur with probability one, and (3) several table updates are needed to generate one table.

6.7.3 Code (Table) Update for $b \neq 00 \dots 0$

EWEMrl with $b \neq 00 \dots 0$ has been proposed to reduce the server overheads. In this version, the assumption (see Assumption 1) is significantly more liberal compared to Property 1 for EWEMrl with $b = 00 \dots 0$. Note that from Assumption 1 there is no fixed point for both $T_{n_{in}}^{(2,0)}$, and $T_{n_{in}}^{(2,1)}$. Thus, the server will check where there exists any x , that is a fixed for both $T_{n_{in}}^{(2,0)}$, and $T_{n_{in}}^{(2,1)}$. If yes, then the server rejects both of them and samples again with another independently sampled key. Note that the probability of the occurrence of this event is $2^{-n_{in}}$. Hence, it is expected that after every $2^{n_{in}}$ updates. Note that the key space will get a negligible reduction, and server overheads will get reduced significantly (only one extra table construction after every $2^{n_{in}}$ rounds).

6.7.4 Communication Cost

In this setup, the server sends the updated table to the client via a secure protocol (for example, TLS). Here, the communication cost is the number of bits transmitted during each update. In our case, the communication cost is the size of the code (table), mentioned in Table 4.

6.8 Inapplicability of Longevity-Attack from [32] Based on the Canonical Representation

The attack based on the canonical representation does not apply to EWEMrl. We will briefly discuss it. Precisely, the longevity attack on Yoroi

- first defines a property P about the tables (see Property 1, [32]) used for Yoroi,
- it next proposes that the tables will be uniquely determined if they satisfy this property (see Proposition 1, [32]), and finally
- it provides an efficient algorithm (see Algorithm 1, in [32]) to detect whether the tables satisfy this property or not.

The detection algorithm (Algorithm 1, in [32]) is the core of this attack, and it exploits the fact that the t least significant bits from the Sboxes are leaked from the tables because they are unmasked (see Line 5, Algorithm 1, in [32]). If no bits of the Sboxes are leaked, then there is no way to detect this property, and the attack won't work. Our design carefully resists this attack by fully masking the Sboxes. Precisely, as mentioned in Section 6.2,

- $T_{n_{in}}^{(1)}$ corresponds to $E_{K^*} \circ S_{K^{(1)}}$ and hence $S_{K^{(1)}}$ is fully masked by E_{K^*} ,
- $T_{n_{in}}^{(2,0)}$ corresponds to $E_{K^*} \circ S_{K^{(2,0)}} \circ D_{K^*}$ and hence $S_{K^{(2,0)}}$ is fully masked by E_{K^*} and D_{K^*} ,
- $T_{n_{in}}^{(2,1)}$ corresponds to $E_{K^*} \circ S_{K^{(2,1)}} \circ D_{K^*}$ and hence $S_{K^{(2,1)}}$ is fully masked by E_{K^*} and D_{K^*} , and
- $T_{n_{in}}^{(3)}$ corresponds to $S_{K^{(3)}} \circ D_{K^*}$ and hence $S_{K^{(3)}}$ is fully masked by D_{K^*} ,

Note that, K^* is independent sampled from $K^{(1)}, K^{(2,0)}, K^{(2,1)}$, and $K^{(3)}$. This depicts that the Sboxes are independent of the tables, and no information about the Sboxes will be leaked from the tables, making the detection process impossible.

6.9 Code (Table) Size of EWEMrl

When $b = 00 \dots 0$, the construction EWEMrl has three tables. When $b \neq 00 \dots 0$, EWEMrl has four tables. Each table is of size $n_{in} \cdot 2^{n_{in}}$ bits.

6.10 A Natural Longevity Secure Extension of EWEMx: EWEMxl

In the design of EWEMxl, one EWEMrl is sandwiched by two EWEMr's, such that the upper and lower parts of the sandwich use independently (independent of the key used in EWEMrl) keyed Sboxes. EWEMxl is structurally similar to EWEMx except that the middle EWEMr is replaced by EWEMrl. Precisely, EWEMxl is defined as

$$\text{EWEMr}_{K_3}^{n_{in}}[r_3, \ell_3, w, \pi_3] \circ \pi \circ \text{EWEMrl}_{K_2}^{n_{in}}[r_2, \ell_2, w, \pi_2, b] \circ \pi \circ \text{EWEMr}_{K_1}^{n_{in}}[r_1, \ell_1, w, \pi_1].$$

Note that the longevity security is provided by the middle EWEMrl layer. The upper and the lower EWEMr layers do not contribute to longevity. They only contribute to obtaining security for one-time leakage (space-hardness and hybrid code-lifting attack). Below, we briefly mention the security of EWEMxl.

6.10.1 Space-hardness Bounds for EWEMxl

In the case of KS, CS, and ACS, EWEMxl achieves at least the same level of space-hardness as EWEMx due to using multiple tables. The number of possible collisions is less than that in EWEMx, which in turn may even provide a better space-hardness bound than EWEMx. Note that, in this case, we are actually depicting that the value of Z in Definition 4 for EWEMxl is lower bounded by that for EWEMx. This means for EWEMxl, the probability of encrypting a random plaintext is upper bounded by the same as for EWEMx. Hence, we are not providing any proof for the space-hardness bounds for EWEMxl.

6.10.2 Hybrid Code Lifting Attack for EWEMxl

The hybrid code-lifting attack of EWEMxl is similar to that of EWEMx, which is secure for any adaptive adversary. EWEMxl achieves at least the same security bound as EWEMx for hybrid code-lifting. Hence, EWEMxl is also secure against hybrid code-lifting attacks against adaptive adversaries.

6.10.3 Longevity for EWEMxl

EWEMxl sandwiches EWEMrl, and the longevity (against non-adaptive and read-only malware) of EWEMxl comes from that of EWEMrl under the attack model as described in Section 1.5. Note that, the upper and lower parts of the sandwich use independently keyed Sboxes and are not updated, and we assume they are leaked after a few updates. Hence, it achieves the same longevity bound as EWEMrl.

6.10.4 Code (Table) Size of EWEMxl

When $b = 00 \dots 0$, EWEMxl needs four tables, and the server overhead is high. For $b \neq 00 \dots 0$, EWEMxl needs five tables, and the server overhead is low. Each table is of size $n_{in} \cdot 2^{n_{in}}$ bits. The extra table from EWEMrl is for the upper and lower parts, and this extra table is independent of the tables of the middle layer of EWEMxl.

7 Black-Box Security Analysis of EWEM

In this section, we discuss the security of EWEM in the black-box setting. The analysis of key-dependent functions, $\mathcal{S}_k$, is given in [8] and it was shown to be secure against differential, linear, and meet-in-the-middle attacks due to the use of the components of well-analyzed AES. Note that, even if we have ℓ -iteration of keyed Sbox, from an attacker's point of view $\mathcal{S}_k^\ell$ can be seen as a single layer as there is no diffusion between these. Thus, in the black-box setting, EWEM instances get reduced to WEM with $r \geq 2$. In the design proposal of WEM [11], the authors provide an analysis of WEM explicitly for $r \geq 2$. However, it is important to note that WEM with $r = 1$ does not provide n -bit security as shown in [11]. In this sense, EWEM instances attain the **minimum** secure value for r . Here, we recall the main results in the black-box setting.

- The best known attack on two round EM is given in [31]. As we have replaced key addition with keyed Sbox, all the best-known attacks on two round EM are not applicable to EWEMr and EWEMrl.
- EWEMr and EWEMrl are secure against generic key recovery attacks due to the use of AES₆. The current best-known attack on AES₆ with secret Sboxes is with complexity 2^{96} . Thus, our design with 12-round AES is key-recovery secure.
- Except for key-recovery, one can think about differential and linear distinguishers as well. In this case, from previous analysis on AES [11], our design with 12 rounds of AES is far from danger.
- We also consider other cryptanalytic attacks, such as impossible differential, higher order differential, and algebraic attacks. We find that none of them work in less than 2^κ complexity, mainly for the use of 12 round AES (also discussed in [11]).

8 Grey-box Security Analysis of EWEM Instances

We now describe the resistance of EWEM instances against Differential Computation Analysis (DCA) and Differential Fault Analysis (DFA).

8.1 Security against Differential Computation Analysis

In [9], Bos et al. proposed a new software-based side-channel attacks denoted by *differential computation analysis* (DCA). DCA observes the software execution of the white-box implementation of AES [12, 25, 34], and analyzes the memory access patterns using PIN and Valgrind. In the case of AES, the software traces contain the physical address of memory demarcated by time. These memory locations are being read or written into, and hence leak the values of inputs to the various tables

that are accessed. This information can be used as the side channel information for key extraction. Precisely, DCA uses the observation that information about the intermediate variable is leaked through the implementation during the algorithms are executed. In fact, these variables may depend on the plaintext and a part of the master key. This is due to the fact that each of the tables depends only on the part of the key (for example, 8, or 16-bits of the key [12, 25, 34]). Hence, a small part of the master key could be extracted using the leakages. We would like to emphasize that the EWEM instances (similar to SPNbox, Yoroï, and several others) use tables containing the master key. Thus, even if the adversary observes the memory access patterns for the tables, the number of such patterns is 2^{128} , which is huge. Hence, DCA will not be effective against the EWEM instances.

8.2 Security against Differential Fault Analysis

In 2015, Sanfelix et al. in [21] proposed differential fault analysis (DFA) on white-box AES and DES. For white-box AES, the attack injects one byte in the 9th MixColumns. For the EWEM instances, each of the tables is a small block cipher, and the internal states of the block ciphers are embedded inside tables. Thus, the attacker can only inject faults at the input of the Sboxes during the encryption. Thus, any fault attack reduces to differential analysis on the small block ciphers corresponding to the Sboxes. Hence, this reduces to a black-box. Since the Sboxes are assumed to be secure against black-box differential analysis, DFA won't work for the EWEM instances.

8.3 Cache Timing Attack

Several techniques are there (Prime+Probe attack [2], and Flush+Reload attacks [3, 22, 35]) to exploit cache information like timing differences between cache hits and misses. In EWEM instances, the keys are embedded inside the keyed Sboxes. They can be implemented with AES-NI instructions and are designed to be executed in constant time, and no cache accesses are done during key-dependent operations. Thus, it would be hard to apply cache timing attacks against black-box implementations of the EWEM instances.

9 Implementation

One of the main design motivations for EWEMr and EWEMrl is to utilize the AES-NI instructions on Intel and AMD processors. π , which is six rounds of AES, can be realized by the `aesenc` instruction. To make the implementation more portable on different platforms (e.g., 32-bit platform), we provide a table-based implementation of EWEMr and report time requirements. Both implementations provide competitive performance. EWEMr¹⁶[2, 8, 8, AES₆] needs 15 cycles per byte (cpb) using AES-NI and 27 cpb for table-based implementation, and outperforms WEM-16 by a factor of 2 and 3, respectively.

For $\chi_K^{\ell, w}$'s implementation, we apply a similar approach used to compute small ciphers as in SPNbox. The computation of S_K uses `aesenc` instructions of one round AES encryption. For $n_{in} = 32$, we compute $AK^i \circ MC_{32} \circ SB$ for a given round key. To achieve this, the `pshufb` instruction is first applied to perform the inverse of the AES shift row, followed by the application of the `aesenc` instruction. For, $n_{in} = 16$, the application of MC_{16} is similar to that of $MC_{32} \times (x, y, 00_x, 00_x)$, i.e., we ignore last 16 bits. We pre-compute S_K using the above approach and implement using a table-based implementation.

Performance Measurement: Performance of EWEMr is compared with corresponding versions of WEM, SPNbox, and SPACE. For longevity, we compare our design EWEMrl with Yoroï. We measure performance in both black-box and white-box settings, averaging each experiment over 100000 repetitions, processing one message at a time. All experiments were conducted on a single core of an Intel(R) Core(TM)

Table 4: Summary of Performance Measurements

Ciphers	Space-hardness		Longevity Secure Yes/No	White-box Setting		Black-box Setting Cycle Per Byte
	KS/CS	ACS		Table Size	Cycle Per Byte	
EWEMr ¹⁶ [2, 8, 8, AES ₆]	(0.687, 128)	Not Secure	No	128 KB	15.85/27.12*	8.62/17.57*
EWEMr ¹⁶ [2, 16, 8, AES ₆]	(0.355, 128)	Not Secure	No	128 KB	24.72/34.76*	8.62/17.57*
EWEMx ¹⁶ [2, 2, 2, 1, 8, 1, 8, AES ₁ , AES ₁ , AES ₆ , AES ₁]	(0.55, 128)	(0.55, 118.10)	No	128 KB	31.65/48.11*	16.23/30.66*
EWEMx ¹⁶ [2, 2, 2, 1, 16, 1, 8, AES ₁ , AES ₁ , AES ₆ , AES ₁]	(0.3245, 128)	(0.3245, 117.76)	No	128 KB	50.34/70.16*	16.23/30.66*
WEM – 16	(1.32, 128)	(2, 112)	No	128 KB	30.76/91.23*	30.76/91.23*
SPACE – 16	(1.1, 128)	(1.14, 119.72)	No	896 KB	280.12	338.19
SPNbox – 16	(1.61, 128)	(1.12, 88.4)	No	128 KB	25.57	25.75
WhiteBlock – 16	(1.78, 128)	(2, 112)	No	2 MB	182.37	105.20
EWEMr ³² [2, 8, 4, AES ₆]	(1.334, 128)	Not Secure	No	16 GB	505.26	85.81
EWEMr ³² [2, 16, 4, AES ₆]	(0.667, 128)	Not Secure	No	16 GB	998.23	175.25
EWEMx ³² [2, 2, 2, 1, 8, 1, 4, AES ₁ , AES ₁ , AES ₆ , AES ₁]	(1.067, 128)	(1.067, 102.05)	No	16 GB	985.76	157.43
EWEMx ³² [2, 2, 2, 1, 16, 1, 4, AES ₁ , AES ₁ , AES ₆ , AES ₁]	(0.593, 128)	(0.593, 101.55)	No	16 GB	1400.11	332.88
SPACE – 32	(1, 128)	(1.14, 103.72)	No	48 GB	2261.13	335.10
SPNbox – 32	(3.20, 128)	(3.20, 105.12)	No	16 GB	450.28	48.54
WhiteBlock – 32	(1.88, 128)	(2, 96)	No	64 GB	1243.60	157.28
EWEMrl ¹⁶ [2, 12, 8, AES ₆ , b] b = 10100011110	(0.465, 128)	Not Secure	Yes	512 KB	30.68	6.62
EWEMrl ¹⁶ [2, 21, 8, AES ₆ , b] b = 00101110001111010001	(0.276, 128)	Not Secure	Yes	512 KB	65.72	6.62
EWEMrl ¹⁶ [2, 27, 8, AES ₆ , b] b = 0000101100001111010010001	(0.221, 128)	Not Secure	Yes	512 KB	89.21	6.62
EWEMrl ¹⁶ [2, 2, 2, 1, 12, 1, 8, AES ₁ , AES ₁ , AES ₆ , AES ₁ , b] b = 10100011110	(0.404, 128)	(0.404, 117.88)	Yes	640 KB	57.49	11.32
EWEMrl ¹⁶ [2, 2, 2, 1, 21, 1, 8, AES ₁ , AES ₁ , AES ₆ , AES ₁ , b] b = 00101110001111010001	(0.266, 128)	(0.266, 117.69)	Yes	640 KB	117.09	11.32
EWEMrl ¹⁶ [2, 2, 2, 1, 27, 1, 8, AES ₁ , AES ₁ , AES ₆ , AES ₁ , b] b = 0000101100001111010010001	(0.225, 128)	(0.225, 117.66)	Yes	640 KB	158.33	11.32
Yoroi – 16	(2, 128)	(2.04, 119.83)	No	384 KB	24.89	24.8
EWEMrl ³² [2, 16, 4, AES ₆ , b] b = 0000000000000000	(0.667, 128)	Not Secure	Yes	48 GB	1783.25	178.09
EWEMrl ³² [2, 16, 4, AES ₆ , b] b = 000000111111000	(0.667, 128)	Not Secure	Yes	64 GB	1995.52	178.09
EWEMrl ³² [2, 18, 4, AES ₆ , b] b = 0000001111110000	(0.593, 128)	Not Secure	Yes	64 GB	2274.93	178.09
EWEMxl ³² [2, 2, 2, 1, 16, 1, 4, AES ₁ , AES ₁ , AES ₆ , AES ₁ , b] b = 0000000000000000	(0.593, 128)	(0.593, 101.55)	Yes	64 GB	2877.44	327.12
EWEMxl ³² [2, 2, 2, 1, 16, 1, 4, AES ₁ , AES ₁ , AES ₆ , AES ₁ , b] b = 000000111111000	(0.593, 128)	(0.593, 101.55)	Yes	80 GB	3752.03	327.12
EWEMxl ³² [2, 2, 2, 1, 18, 1, 4, AES ₁ , AES ₁ , AES ₆ , AES ₁ , b] b = 0000001111110000	(0.534, 128)	(0.534, 101.50)	Yes	80 GB	4251.22	327.12
Yoroi – 32	(2, 128)	(2.04, 103.84)	No	48 GB	936.47	68.82

We use AES-NI to implement AES whenever possible. However, we also do table-based implementation of EWEMr and WEM. * denotes the result of the table-based implementation.

i7-1165G7 2.80GHz CPU with 32 GB DDR5 RAM. The machine includes 192 KB L1 data cache, 128 KB L1 instruction cache, 5 MB L2 cache, and 12 MB L3 cache. Whenever possible, we use AES-NI instructions to implement WEM, SPACE, Yoroi, and SPNbox. Additionally, we implement constant-time field multiplication for the linear layers of SPNbox and Yoroi as suggested in [8].

Here, we also briefly discuss the decryption performance of EWEMr and EWEMrl. Decryption in both cases requires the application of the inverse of the AES round function. Fortunately, modern processors equipped with the AES-NI extension set provide the `aesdec` instruction, which performs this inverse transformation efficiently in hardware. It is important to highlight that the performance characteristics of the `aesdec` are similar to `aesenc`. Official documentation from Intel and AMD indicates that both instructions typically exhibit similar latency and throughput on supported architectures. As a result, the cost of decryption closely mirrors that of encryption for both EWEMr and EWEMrl.

The results for EWEMr and EWEMrl encryption are presented in Table 4, with other ciphers for comparison, and EWEMr offers comparable performance. EWEMrl as well provides longevity and comparable performance with Yoroi.

Table 4 shows that EWEMr performs competitively with other ciphers in both white-box and black-box settings. While EWEMr uses more table look-ups, its fewer permutation layers enhance efficiency. We compare WEM-16 with EWEMr¹⁶ in the white-box setting, due to similarity. For both, we implement the AES permutation using AES-NI and tables. In the table-based implementation, EWEMr¹⁶[2, 8, 8, AES₆] and EWEMr¹⁶[2, 16, 8, AES₆] are nearly three times more efficient than WEM-16. Using AES-NI, EWEMr¹⁶[2, 8, 8, AES₆] is twice as efficient as WEM-16, while EWEMr¹⁶[2, 16, 8, AES₆] performs similarly to WEM-16.

10 Conclusion

The paper first generalizes WEM to EWEMr and uses a new proof idea for space-hardness in the non-adaptive setting using collision chains. EWEMr is also shown

to be secure against a hybrid code-lifting attack in the same non-adaptive setting. EWEMr is extended to EWEMx, a space-hardness and hybrid code-lifting attack secure cipher in the adaptive setting. Next, we propose the main contribution EWEMrl, the first white-box cipher achieving longevity against non-adaptive read-only adversaries. It is instantiated with six versions based on the choice of Sboxes. One instance, EWEMrl³²[2, 16, 4, AES₆, 00000000000000], achieves longevity, low code size, but with a complex code update process. All other instances with $b \neq 0$ achieve longevity and a higher code size, but with a highly efficient code update process. Precisely, all EWEMrl instances achieve high longevity and space-hardness against non-adaptive, read-only adversaries. We naturally extend EWEMrl to EWEMxl, achieving high longevity against non-adaptive adversaries and high space-hardness along with security against hybrid code-lifting attacks against stronger adaptive adversaries. Finally, we provide implementation details for all instances on Intel processors.

Acknowledgements. We are highly thankful to the anonymous reviewers for their fruitful comments and suggestions. The second author, Shibam Ghosh, is supported by the ERC grant ReSCALE (number 101041545). The third author, Takanori Isobe, is supported by JST CRONOS, Japan, and JSPS KAKENHI, Grant Number JP24H00696.

References

- [1] Adi Akavia, Shafi Goldwasser, and Carmit Hazay. Distributed public key schemes secure against continual leakage. In *Proceedings of the 2012 ACM symposium on Principles of distributed computing*, pages 155–164, 2012.
- [2] Gorka Irazoqui Apecechea, Thomas Eisenbarth, and Berk Sunar. S\$a: A shared cache attack that works across cores and defies VM sandboxing - and its application to AES. In *2015 IEEE Symposium on Security and Privacy, SP 2015, San Jose, CA, USA, May 17-21, 2015*, pages 591–604. IEEE Computer Society, 2015.
- [3] Gorka Irazoqui Apecechea, Mehmet Sinan Inci, Thomas Eisenbarth, and Berk Sunar. Wait a minute! A fast, cross-vm attack on AES. In Angelos Stavrou, Herbert Bos, and Georgios Portokalidis, editors, *Research in Attacks, Intrusions and Defenses - 17th International Symposium, RAID 2014, Gothenburg, Sweden, September 17-19, 2014. Proceedings*, volume 8688 of *Lecture Notes in Computer Science*, pages 299–319. Springer, 2014.
- [4] Fabrice Benhamouda, Akshay Degwekar, Yuval Ishai, and Tal Rabin. On the local leakage resilience of linear secret sharing schemes. *Journal of Cryptology*, 34:1–65, 2021.
- [5] Olivier Billet, Henri Gilbert, and Charaf Ech-Chatbi. Cryptanalysis of a white box AES implementation. In Helena Handschuh and M. Anwar Hasan, editors, *Selected Areas in Cryptography, 11th International Workshop, SAC 2004, Waterloo, Canada, August 9-10, 2004, Revised Selected Papers*, volume 3357 of *Lecture Notes in Computer Science*, pages 227–240. Springer, 2004.
- [6] Alex Biryukov, Charles Bouillaguet, and Dmitry Khovratovich. Cryptographic schemes based on the ASASA structure: Black-box, white-box, and public-key (extended abstract). In Palash Sarkar and Tetsu Iwata, editors, *Advances in Cryptology - ASIACRYPT 2014 - 20th International Conference on the Theory and Application of Cryptology and Information Security, Kaoshiung, Taiwan, R.O.C., December 7-11, 2014. Proceedings, Part I*, volume 8873 of *Lecture Notes in Computer Science*, pages 63–84. Springer, 2014.
- [7] Andrey Bogdanov and Takanori Isobe. White-box cryptography revisited: Space-hard ciphers. In Indrajit Ray, Ninghui Li, and Christopher Kruegel, editors,

- Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security, Denver, CO, USA, October 12-16, 2015*, pages 1058–1069. ACM, 2015.
- [8] Andrey Bogdanov, Takanori Isobe, and Elmar Tischhauser. Towards practical whitebox cryptography: Optimizing efficiency and space hardness. In Jung Hee Cheon and Tsuyoshi Takagi, editors, *Advances in Cryptology - ASIACRYPT 2016 - 22nd International Conference on the Theory and Application of Cryptology and Information Security, Hanoi, Vietnam, December 4-8, 2016, Proceedings, Part I*, volume 10031 of *Lecture Notes in Computer Science*, pages 126–158, 2016.
 - [9] Joppe W. Bos, Charles Hubain, Wil Michiels, and Philippe Teuwen. Differential computation analysis: Hiding your white-box designs is not enough. In Benedikt Gierlichs and Axel Y. Poschmann, editors, *Cryptographic Hardware and Embedded Systems - CHES 2016 - 18th International Conference, Santa Barbara, CA, USA, August 17-19, 2016, Proceedings*, volume 9813 of *Lecture Notes in Computer Science*, pages 215–236. Springer, 2016.
 - [10] Julien Bringer, Hervé Chabanne, and Emmanuelle Dottax. White box cryptography: Another attempt. *IACR Cryptol. ePrint Arch.*, page 468, 2006.
 - [11] Jihoon Cho, Kyu Young Choi, Itai Dinur, Orr Dunkelman, Nathan Keller, Dukjae Moon, and Aviya Veidberg. WEM: A new family of white-box block ciphers based on the even-mansour construction. In *Topics in Cryptology - CT-RSA 2017*, volume 10159 of *LNCS*, pages 293–308. Springer, 2017.
 - [12] Stanley Chow, Philip A. Eisen, Harold Johnson, and Paul C. van Oorschot. White-box cryptography and an AES implementation. In Kaisa Nyberg and Howard M. Heys, editors, *Selected Areas in Cryptography, 9th Annual International Workshop, SAC 2002, St. John's, Newfoundland, Canada, August 15-16, 2002. Revised Papers*, volume 2595 of *Lecture Notes in Computer Science*, pages 250–270. Springer, 2002.
 - [13] Stanley Chow, Philip A. Eisen, Harold Johnson, and Paul C. van Oorschot. A white-box DES implementation for DRM applications. In Joan Feigenbaum, editor, *Security and Privacy in Digital Rights Management, ACM CCS-9 Workshop, DRM 2002, Washington, DC, USA, November 18, 2002, Revised Papers*, volume 2696 of *Lecture Notes in Computer Science*, pages 1–15. Springer, 2002.
 - [14] Francesco Davì, Stefan Dziembowski, and Daniele Venturi. Leakage-resilient storage. In Juan A. Garay and Roberto De Prisco, editors, *Security and Cryptography for Networks, 7th International Conference, SCN 2010, Amalfi, Italy, September 13-15, 2010. Proceedings*, volume 6280 of *Lecture Notes in Computer Science*, pages 121–137. Springer, 2010.
 - [15] Cécile Delerablée, Tancrede Lepoint, Pascal Paillier, and Matthieu Rivain. White-box security notions for symmetric encryption schemes. In Tanja Lange, Kristin E. Lauter, and Petr Lisonek, editors, *Selected Areas in Cryptography - SAC 2013 - 20th International Conference, Burnaby, BC, Canada, August 14-16, 2013, Revised Selected Papers*, volume 8282 of *Lecture Notes in Computer Science*, pages 247–264. Springer, 2013.
 - [16] Mastercard Developer. Mastercard Mobile Payment SDK Security Guide. <https://developer.mastercard.com/media/32/b3/b6a8b4134e50bfe53590c128085e/mastercard-mobile-payment-sdk-security-guide-v2.0.pdf>, 2017. Accessed: 2025-07-01.
 - [17] Yevgeniy Dodis, Allison B. Lewko, Brent Waters, and Daniel Wichs. Storing secrets on continually leaky devices. In Rafail Ostrovsky, editor, *IEEE 52nd Annual Symposium on Foundations of Computer Science, FOCS 2011, Palm Springs, CA, USA, October 22-25, 2011*, pages 688–697. IEEE Computer Society, 2011.

- [18] Morris Dworkin. Sha-3 standard: Permutation-based hash and extendable-output functions, 2015-08-04 2015.
- [19] Antonio Faonio and Jesper Buus Nielsen. Fully leakage-resilient codes. In *IACR International Workshop on Public Key Cryptography*, pages 333–358. Springer, 2017.
- [20] Pierre-Alain Fouque, Pierre Karpman, Paul Kirchner, and Brice Minaud. Efficient and provable white-box primitives. In Jung Hee Cheon and Tsuyoshi Takagi, editors, *Advances in Cryptology - ASIACRYPT 2016 - 22nd International Conference on the Theory and Application of Cryptology and Information Security, Hanoi, Vietnam, December 4-8, 2016, Proceedings, Part I*, volume 10031 of *Lecture Notes in Computer Science*, pages 159–188, 2016.
- [21] Eloi Sanfeliu Gonzalez, Cristofaro Mune, and J. de Haas. Unboxing the white-box practical attacks against obfuscated ciphers. In *Black Hat Europe 2015*. IEEE, 2015.
- [22] Daniel Gruss, Raphael Spreitzer, and Stefan Mangard. Cache template attacks: Automating attacks on inclusive last-level caches. In Jaeyeon Jung and Thorsten Holz, editors, *24th USENIX Security Symposium, USENIX Security 15, Washington, D.C., USA, August 12-14, 2015*, pages 897–912. USENIX Association, 2015.
- [23] J. Alex Halderman, Seth D. Schoen, Nadia Heninger, William Clarkson, William Paul, Joseph A. Calandrino, Ariel J. Feldman, Jacob Appelbaum, and Edward W. Felten. Lest we remember: Cold boot attacks on encryption keys. In Paul C. van Oorschot, editor, *Proceedings of the 17th USENIX Security Symposium, July 28-August 1, 2008, San Jose, CA, USA*, pages 45–60. USENIX Association, 2008.
- [24] Yael Tauman Kalai and Leonid Reyzin. A survey of leakage-resilient cryptography. In *Providing Sound Foundations for Cryptography: On the Work of Shafi Goldwasser and Silvio Micali*, pages 727–794. ACM, 2019.
- [25] Mohamed Karroumi. Protecting white-box AES with dual ciphers. In Kyung Hyune Rhee and DaeHun Nyang, editors, *Information Security and Cryptology - ICISC 2010 - 13th International Conference, Seoul, Korea, December 1-3, 2010, Revised Selected Papers*, volume 6829 of *Lecture Notes in Computer Science*, pages 278–291. Springer, 2010.
- [26] Yuji Koike and Takanori Isobe. Yoroi: Updatable whitebox cryptography. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2021(4):587–617, 2021.
- [27] Hamilton E. Link and William D. Neumann. Clarifying obfuscation: Improving the security of white-box DES. In *International Symposium on Information Technology: Coding and Computing (ITCC 2005), Volume 1, 4-6 April 2005, Las Vegas, Nevada, USA*, pages 679–684. IEEE Computer Society, 2005.
- [28] Yoni De Mulder, Peter Roelse, and Bart Preneel. Cryptanalysis of the xiao - lai white-box AES implementation. In Lars R. Knudsen and Huapeng Wu, editors, *Selected Areas in Cryptography, 19th International Conference, SAC 2012, Windsor, ON, Canada, August 15-16, 2012, Revised Selected Papers*, volume 7707 of *Lecture Notes in Computer Science*, pages 34–49. Springer, 2012.
- [29] Yoni De Mulder, Brecht Wyseur, and Bart Preneel. Cryptanalysis of a perturbed white-box AES implementation. In Guang Gong and Kishan Chand Gupta, editors, *Progress in Cryptology - INDOCRYPT 2010 - 11th International Conference on Cryptology in India, Hyderabad, India, December 12-15, 2010. Proceedings*, volume 6498 of *Lecture Notes in Computer Science*, pages 292–310. Springer, 2010.
- [30] NIST. CVE-2014-0160: Heartbleed Vulnerability. <https://nvd.nist.gov/vuln/detail/CVE-2014-0160>, 2014. Accessed: 2025-07-01.

- [31] Tyge Tiessen, Lars R. Knudsen, Stefan Kölbl, and Martin M. Lauridsen. Security of the AES with a secret s-box. In *FSE*, volume 9054 of *Lecture Notes in Computer Science*, pages 175–189. Springer, 2015.
- [32] Yosuke Todo and Takanori Isobe. Hybrid code lifting on space-hard block ciphers application to yoroi and spnbox. *IACR Trans. Symmetric Cryptol.*, 2022(3):368–402, 2022.
- [33] Brecht Wyseur, Wil Michiels, Paul Gorissen, and Bart Preneel. Cryptanalysis of white-box DES implementations with arbitrary external encodings. In Carlisle M. Adams, Ali Miri, and Michael J. Wiener, editors, *Selected Areas in Cryptography, 14th International Workshop, SAC 2007, Ottawa, Canada, August 16-17, 2007, Revised Selected Papers*, volume 4876 of *Lecture Notes in Computer Science*, pages 264–277. Springer, 2007.
- [34] Yaying Xiao and Xuejia Lai. A secure implementation of white-box aes. In *2009 2nd International Conference on Computer Science and its Applications*, pages 1–6. IEEE, 2009.
- [35] Yuval Yarom and Katrina Falkner. FLUSH+RELOAD: A high resolution, low noise, L3 cache side-channel attack. In Kevin Fu and Jaeyeon Jung, editors, *Proceedings of the 23rd USENIX Security Symposium, San Diego, CA, USA, August 20-22, 2014*, pages 719–732. USENIX Association, 2014.